MSc Artificial Intelligence
Master Thesis

Whisper to Speech Conversion Using Diffusion Models

by
Jonathan Marten van Westendorp
11325747

June 30, 2022

48EC
1/11/2021 - 1/7/2022

Supervisor:
Mario Giulianelli

Examiner:
Dr. Raquel Fernández Rovira

Universiteit van Amsterdam

Acknowledgments

Special thanks to Bram Kooiman and Akash Raj of Whispp and Mario Giulianelli of the University of Amsterdam, whose daily advise, guidance, and dedication to this project could not have been missed. I would also like to thank Joris Castermans of Whispp for giving me the inspiring opportunity to transfer my knowledge to the audio domain.

Abstract

Whispering is often the only way to communicate without problems for people who stutter or suffer from vocal cord impairment. This group of people could greatly benefit from a whisper to speech conversion model that gives them back their own natural voice. We propose a novel approach to whisper to speech conversion, based on diffusion models. Diffusion models do not require adversarial training, making them more stable than generative adversarial networks. They can also sample from the true posterior with high fidelity through a process called Langevin Dynamics. We show that our diffusion-based method achieves intelligibility that is highly competitive with the state of the art solution employed by Whispp, while being superior in terms of naturalness. We conclude by presenting our method in a streaming setting, paving the way for future inference speed optimization.

Contents

1 Introduction **1**

2 Background **3**
2.1 Sound . . . 3
2.2 Phonemes . . . 4
2.3 Spectrograms . . . 4
2.4 Vocoders . . . 6
2.5 Whispered Speech vs. Voiced Speech . . . 7
2.6 Aligning Whispered and Voiced Spectrograms . . . 7
2.6.1 Dynamic Time Warping . . . 7
2.6.2 Montreal Forced Alignment . . . 8
2.7 Speaker Variation . . . 8
2.8 Streaming . . . 9

3 Related work **10**
3.1 Vocoders . . . 10
3.2 Voice Conversion . . . 10
3.3 Whisper to Speech . . . 11
3.3.1 Early Approaches . . . 11
3.3.2 State of the Art . . . 12
3.3.3 Whispp's Approach . . . 12
3.4 Diffusion-Based Modeling . . . 12
3.4.1 Diffusion Probabilistic Models . . . 12
3.4.2 Langevin Dynamics . . . 15
3.4.3 Score-Based Models . . . 15
3.4.4 Stochastic Differential Equations . . . 16
3.4.5 Applications . . . 18

4 Method **21**
4.1 Spectrogram to Waveform . . . 21
4.1.1 WaveGrad . . . 21
4.1.2 DiffWave . . . 22
4.2 Spectrogram to Spectrogram . . . 23
4.2.1 DiffVC . . . 23
4.3 Streaming . . . 26

5 Experiments **28**
5.1 Dataset . . . 28
5.2 Hyperparameters . . . 29
5.3 Model Evaluation . . . 29

5.3.1 Objective Evaluation . . . 29
5.3.2 Subjective Evaluation . . . 30
5.4 Streaming . . . 32

6 Results **33**
6.1 Automatic Evaluation . . . 33
6.1.1 Spectrogram to Waveform . . . 33
6.1.2 Spectrogram to Spectrogram . . . 33
6.2 Human Evaluation . . . 36
6.3 Streaming . . . 38

7 Conclusion **39**
7.1 Future Research . . . 40

A **46**
A.1 ELBO Definition . . . 46
A.2 ELBO Reparameterization . . . 47

B **48**
B.1 The effect of dropout and batch normalization on DiffVC-W2P . . . 48

Chapter 1

Introduction

Vocal communication is one of the most important ways of connecting with others and exchanging information. Speech disorders that result in the inability to speak clearly can therefore be highly limiting for people that suffer from them. Stuttering, for example, can strongly impair confidence and intelligibility. Other vocal disorders include vocal cord paralysis, spasmodic dysphonia and other conditions that limit functionality of the vocal cord. However, people who suffer from vocal cord problems often have no issue whispering [Ingham et al., 2009]. This is because vocal cords are redundant for whisper-like sounds. A method for converting whisper to phonated speech could therefore be greatly beneficial for people who suffer from speech disorders. Such a method could also be valuable in situations where privacy is a concern. Not having to speak loudly for the receiver to hear could help guarantee the speaker's confidentiality. Military applications come to mind, where discretion is often very important.

The research presented in this thesis has been carried out at Whispp[1], a start-up that specializes in speech applications for people with voice disorders. Whispp aspires to launch an application that translates a person's whispered voice to their own natural spoken voice without delay, so that calls can be made without the hindrance of stuttering or other disorders. Even though Whispp has made substantial progress, the final product still needs research and development. Whisper to speech (W2S) conversion is not a new field of research. The first approaches were based on Gaussian mixture models (GMMs) [Toda and Shikano, 2005]. When deep neural methods such as Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) were introduced, they were also employed for W2S [Malaviya et al., 2020, Kim et al., 2017]. The current state of the art approach is GAN-based and relies on attention modules to better understand temporal relations [Gao et al., 2021]. Although this approach is better than previous methods, the synthetic samples can be easily distinguished from normal speech, which is undesirable for a production model. In this work, we explore a recently established generative model family, diffusion models, as a new approach for W2S conversion.

Score-based generative models (SGMs) [Song and Ermon, 2020] and diffusion probabilistic models (DPMs) [Sohl-Dickstein et al., 2015], which can both be defined generically as *diffusion models*, were recently shown to be powerful solutions for a wide range of tasks such as shape generation [Cai et al., 2020], music generation [Mittal et al., 2021], unconditional image generation [Song and Ermon, 2020, Sohl-Dickstein et al., 2015]—where they outperform very competitive models such as generative adversarial networks (GANs) [Dhariwal and Nichol, 2021, Song et al., 2021]—, text-conditional image synthesis [Ramesh et al., 2022, Saharia et al., 2022], natural text generation [Li et al., 2022] and voice encoding [Chen et al., 2020, Kong et al.,

[1] https://www.whispp.com/

2021, Popov et al., 2021a]. Both model families iteratively corrupt data samples by adding noise and then learn to reverse this process by learning to predict the noise that was added in each iteration. The resulting generative model can sample new data points by starting from a prior distribution of the added noise and gradually moving to the true posterior distribution through a process called Langevin Dynamics. Diffusion models are not prone to overfitting, do not require adversarial training and can efficiently sample from the true posterior distribution. It will become evident later that score-based generative models and diffusion probabilistic models are theoretically similar. We will use *diffusion models* as an umbrella term for both model types throughout this work. We will also use the adverbial object *diffusion-based* to denote a relation with diffusion models.

Diffusion models have been shown to be able to effectively restore degraded speech [Zhang et al., 2021]. Restoring degraded speech is similar to W2S conversion as whispered speech can be viewed as degraded phonated speech. The fact that diffusion models already performed well on a task that is similar to W2S, combined with the proven effectiveness of diffusion models in numerous related fields such as voice encoding and natural text generation, convinced us that the success of diffusion models will extend to the complex domain of W2S. In this thesis, we will investigate whether and how accurately diffusion models can convert whispered speech to phonated speech compared to state of the art models. Two approaches will be followed and compared: (i) an end-to-end approach, where a translation is learned between a whispered mel-spectrogram and a voiced waveform directly; (ii) a spect-to-spect approach, where a mapping is learned between a whispered mel-spectrogram and a spoken mel-spectrogram. The synthetic spoken mel-spectrogram is converted into a waveform using a high quality speech synthesis model.

We will adapt diffusion-based architectures for W2S. The results will be evaluated and compared against a baseline provided by Whispp using generally accepted metrics such as word error rate (WER), character error rate (CER) and a human listening test. Afterwards, we will investigate how well our diffusion-based W2S solution can be used in a streaming setting. We found that our best diffusion model produces synthetic samples that are evidently more natural and often more intelligible than those produced by the baseline model, demonstrating that diffusion models are very well fit for W2S conversion. Our best model can also be adapted for streaming purposes. However, the current model architecture has to be adapted to perform fast enough for real-time applications.

Chapter 2

Background

This chapter will introduce the audio processing concepts and techniques that will be relevant throughout the thesis. We will start with basic concepts in audio-processing and gradually move on to complex audio representations, alignment techniques and vocoders.

2.1 Sound

The word *sound* describes the changes in air pressure our eardrums pick up over time. This information can be represented as a one-dimensional array containing temporal barometric values, also called a waveform. Figure 2.1 visualizes an example of a spoken utterance. The amount of *samples* representing one second of audio information is known as the *sample rate*, measured in Hertz (Hz). As a lot of values are needed to represent a waveform of just one second, valid sample rates are greater than 10kHz, with the generally accepted default at 44.1kHz. This is not an arbitrary number. Humans are known to have an audible frequency range of 20Hz to 20kHz [Rossing, 2007] and to make sure every human-audible sound can be digitized, a sample rate at least twice as big as the maximum a human can hear should be chosen (Nyquist rate) [Nyquist, 1928].

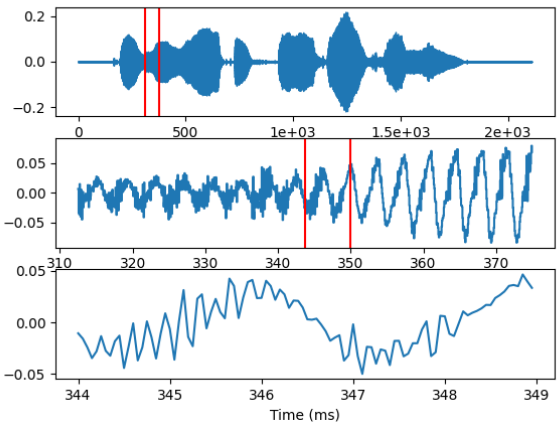


Figure 2.1: Waveform with a sample rate of 16kHz. The second waveform is a segment of the first waveform, and the third waveform is a segment of the second waveform.

2.2 Phonemes

Frequency information over time is not the only important aspect of speech. A randomly generated waveform does not necessarily convey semantic information. It is the combination of *phonemes*, the units of sound that make up words, that can be interpreted by humans. There are multiple ways to display them. The International Phonetic Alphabet (IPA) uses a mix of Latin and Greek characters to display the sounds as accurately as possible. The ARPAbet, developed by Advanced Research Projects Agency (ARPA) uses only ASCII characters to represent phonemes. Table 2.1 shows some example words broken down in the parts that make them up. Both the IPA and the ARPAbet were designed to display every possible sound that could make up a word. The IPA differs from the ARPAbet by being applicable for any language. The ARPAbet was designed only for American English which can differ a lot from British English in terms of accent. For this reason, 'AH' in the ARPAbet can encode 'ə' in the IPA, even though 'AH' originally only directly matches 'ʌ'. Pronunciation can obviously influence this. A benefit of the ARPAbet is that it only uses ASCII characters. This makes it easily adoptable for computational purposes.

Word	**Syllable**	**Phoneme (IPA)**	**Phoneme (ARPAbet)**
home	home	ˈhoʊm	HH-OW-M
hello	he-llo	həˈloʊ	HH-AH-L-OW
elephant	e-le-phant	ˈɛləfənt	EH-L-AH-F-AH-N-T
amazing	a-ma-zing	əˈmeɪzɪŋ	AH-M-EY-Z-IH-NG

Table 2.1: Word to phoneme breakdown.

We will see later that phonemes are of great importance when synthesizing waveforms. To find the exact time a phoneme occurs within a waveform, an alignment algorithm such as the Montreal Forced Aligner (MFA) [McAuliffe et al., 2017] can be employed. MFA uses a combination of probabilistic models and sound files with corresponding transcriptions to find the phonemes within a sound file and the times they occur. Section 2.6 will revisit alignment techniques in more depth.

2.3 Spectrograms

Although the raw audio waveform is the purest representation of sound, it is not often used as input for (neural) models or other analysis methods directly. The large number of samples needed for a short waveform and the fact that those entries hardly convey any semantic information makes the raw audio waveform undesirable to work with. To more effectively analyze the information conveyed in a waveform, we utilize the fact that waveforms are essentially a combination of sine and cosine functions. The Fourier transform extracts these functions to find the separate frequencies and their amplitudes present in the waveform [Bracewell and Bracewell, 1986]. This representation is called the *spectrum*. An efficient algorithm typically used for extracting the spectrum of a waveform is called fast Fourier transform (FFT). Waveforms containing speech information are non periodic, meaning that they do not repeat over time. To represent a non periodic waveform, we need to compute FFT multiple times on overlapping windows in the waveform. This is the short-time Fourier transform (STFT) of a waveform and the resulting representation is called a *spectrogram*. Equation 2.1 shows the

STFT's mathematical definition.

$$X_{fm} = STFT_{fm}(\mathbf{x}) = \sum_{n=m}^{m+N} w_n x_n e^{i\phi n} \tag{2.1}$$

where

$$\sum_{n=m}^{m+N} x_n e^{i\phi n} \tag{2.2}$$

is the discrete Fourier transform and

$$\phi = \frac{2\pi f}{N}.$$

X_{fm} is the presence of frequency f at time m and $\mathbf{x}$ is the input signal. A windowing function w such as the Hamming window defines windows of size N of $\mathbf{x}$ that are transformed using FFT. The overlap between windows is controlled by a parameter called *hop size* or *hop length* (h). The hop length is the space between the start of the current window and the start of a successive window. This overlap ensures that no information is lost at the borders of windows. Figure 2.2 visualizes this concept[1]. The origin of imaginary unit i in the Fourier transform can be found by inspecting Euler's formula:

$$e^{i\phi n} = \cos(\phi n) + i\sin(\phi n).$$

The complex value $w_n x_n e^{i\phi n}$ is the sum of how much $w_n x_n$ agrees with a perfect cosine with frequency f and how much it agrees with a perfect sine with frequency f.

We denote the *complex* spectrogram that results as $X \in \mathbb{C}^{F\times T}$. The complex phase information is typically discarded by taking the absolute value of the complex numbers—yielding a *magnitude* spectrogram, often denoted as $A \in \mathbb{R}^{F\times T}$. We also follow this approach in this work.

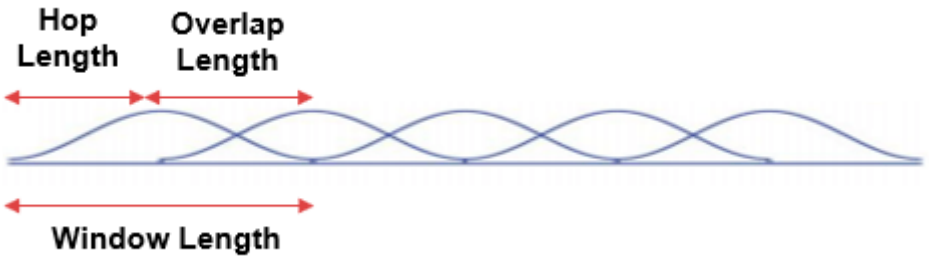


Figure 2.2: A visualization windowing function w. The hop length (h) is the space between the start of the current window and the start of a successive window, the window length (N) is the actual length of a window and the overlap length is the amount of overlap between windows, not parameterized here.

Evaluating the windows in a spectrogram consecutively allows us to effectively analyze the change in frequencies that are also inherent to the one-dimensional waveform. This process is similar to how humans perceive audio, as humans are also able to perceive multiple frequencies at once. However, the frequency axis of the spectrogram is linearly scaled, which does not correspond to how humans perceive change in frequency. The mel-scale [Volkmann et al., 1937] aims to make the measurement of pitch more realistic by transforming the linear frequency axis of the standard spectrogram to a subjective scale that aims to have equal steps where humans also perceive equal steps. Additionally, the mel-scale can reduce the dimensionality of the

[1]Image from https://nl.mathworks.com/help/dsp/ref/dsp.stft.html

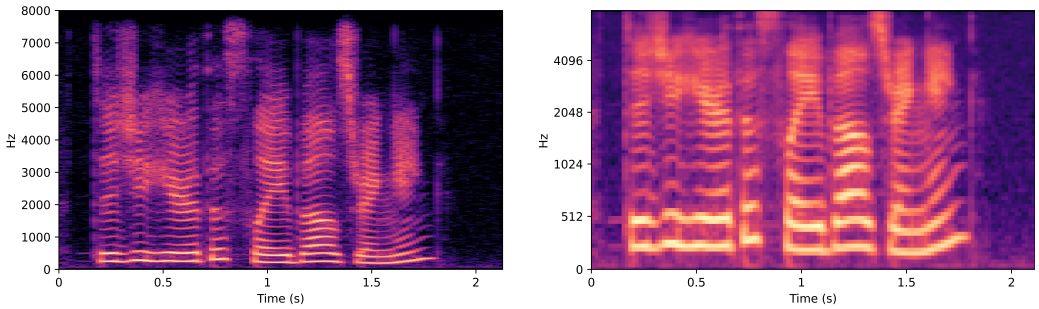


Figure 2.3: A spectrogram with linear y-axis (left) and the same spectrogram with its y-axis converted to the subjective mel-scale (right). The bright spots show the presence of a certain frequency at a certain point in time. The features in the right spectrogram are clearly enhanced compared to the left spectrogram. The transcription reads: "Did you hear the sleigh bells ringing?". We will use mel-spectrograms as the primary representation of audio throughout this work.

spectrogram without losing too much information. The resulting axis resembles a logarithmic function that is linear below 1kHz. Figure 2.3 shows an example of a linear spectrogram and the same spectrogram with y-axis converted to mel-scale. The main benefit of spectrograms is that the information they contain is considerably more obvious compared to raw audio waveforms. This makes relations between input features easier to learn for machine learning algorithms. Furthermore, STFT is very efficient so spectrograms can be quickly computed where needed.

2.4 Vocoders

The derivations given in Eq. 2.1 show how the short-time Fourier transform (STFT) is used to generate spectrograms. These spectrograms are very useful as input for deep learning models, but one cannot *listen* to a spectrogram directly. Consequently, spectrograms that were not synthesized from a waveform but, e.g., as outputs from a neural model cannot be listened to. A means is thus needed by which a waveform can be derived from a mel-spectrogram. This is typically called audio synthesis. Vocoders are often used for this. The term "vocoder" can denote multiple audio related concepts. Its original use was to compress voice information to save bandwidth [Gold et al., 2011], but since the growing popularization of synthetic music, vocoders have also been applied to musicalize voice. In this thesis, we use the term vocoder for an algorithm that converts a spectrogram to a waveform. A problem here is that the (complex) phase information is discarded as this information is deemed redundant for most analytic operations. This phase information is required for accurately performing the inverse STFT (iSTFT). Another problem is the transformation of the y-axis to the mel-scale. This mel-scale typically has fewer dimensions than the linear spectrogram, resulting in information loss. Although the information loss cannot be fully reversed, the linear scale can be regained with multiple methods: (i) The matrix used for transforming to the mel-scale can be transposed and multiplied with the spectrogram; (ii) The pseudo-inverse of the mel matrix can be multiplied with the spectrogram. This generally works better; (iii) The best approach is to solve a non–linear least square problem to find the original linear scale. The Griffin-Lim algorithm (GL) aims to solve the problem of missing phase information by initializing it randomly and then alternating STFT and iSTFT to optimize the phase information [Griffin and Lim, 1983]. Generally, an improvement is seen after 20 iterations. GL can be regarded as a type of vocoder, as it synthesizes audio from other signals, i.e. spectrograms in our case. Although GL is very fast and produces adequate results, it does not generate flawless audio. Neural generative vocoders have shown to produce results that are almost indistinguishable from the source audio [van den Oord et al., 2016, Kumar et al., 2019, Kong et al., 2020, Chen et al., 2020, Kong et al., 2021].

These models also do not require the mel-scale to be reversed to a linear scale first. Section 3.1 will provide a more detailed overview of neural vocoders.

2.5 Whispered Speech vs. Voiced Speech

In literature, different expressions are used to describe whispered or voiced speech. In this thesis, voiced, phonated and spoken will be used interchangeably, referring to the same speech characteristics. Non-audible murmur (NAM) is a lesser-know category of speech that has also been studied as something that could be transformed into voiced speech. NAM can generally not be perceived by human ears and thus a special microphone is needed. As the focus of this project is whisper to speech conversion, NAM will only be mentioned when relevant.

Translating a whispered utterance to a spoken utterance is not an easy task. The most important reason is the fact that most, if not all frequency/pitch information (often denoted as F_0) is missing in a whispered utterance. Some other less visible but equally important differences are vowel-duration, energy level—whispered speech often has about 20dB less energy—and formant frequencies, frequency peaks in the spectrum which have a high degree of energy [Houle and Levi, 2020]. This is visualized in Figure 2.4. The whispered spectrogram on the left shows no sharp bright lines compared to the voiced spectrogram on the right. Inferring the frequency information from a spectrogram with no pitch is thus a shot in the dark without any additional knowledge. Furthermore, we can clearly see that the spectrograms differ in length, even though they are visualizations of the same utterance. This is due to both the difference in vowel duration between speech types and a general speaking speed difference between recordings. This lack of alignment can be problematic for W2S conversion, but fortunately, there are effective techniques for aligning spectrograms that will be discussed in Section 2.6.

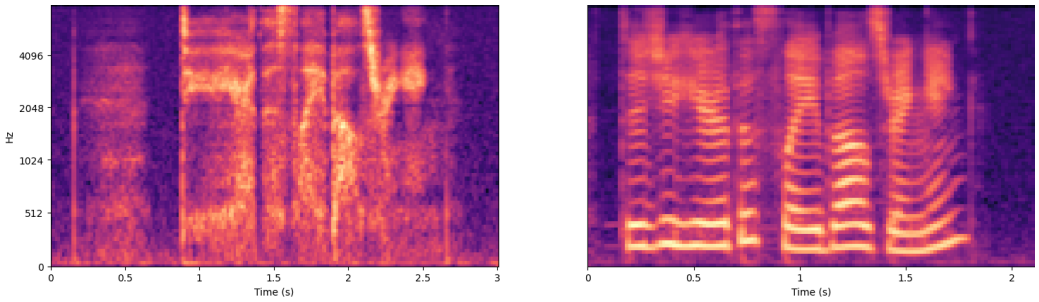


Figure 2.4: Spectrograms of the same utterance whispered (left) and spoken (right).

2.6 Aligning Whispered and Voiced Spectrograms

As discussed in the previous section, the misalignment of a whisper-spoken spectrogram pair is unfavorable. Fortunately, techniques exist that allow us to align two spectrograms so that the phonemes in them are uttered at the same point in time. Two examples are dynamic time warping (DTW) and Montreal forced alignment (MFA).

2.6.1 Dynamic Time Warping

The objective of DTW is to find matches between points in any paired time-series data. The matches are made following four rules:

1. Every index in the first sequence must match one or more indices in the second sequence and vice versa.

2. The first index in the first sequence must match the first index in the second sequence.
3. The last index in the first sequence must match the last index in the second sequence.
4. Indices are mapped increasingly. Indices $j > i$ in the first sequence can only be mapped to $l > k$ in the second sequence if j is matched to l and i is matched to k, not the other way around.

A match is accepted when all rules are followed and the cost of a match is minimal. The cost is equal to the absolute difference of values at time point t in a spectrogram. To align a spectrogram using these matches, the values in the one spectrogram are *warped*, meaning stretched or compressed, to ensure the matches occur at the same point in time. Figure 2.6 shows a whispered spectrogram that has been warped to match the voiced utterance.

2.6.2 Montreal Forced Alignment

On the other hand, Montreal Forced Alignment (MFA) is a tool that can align audio to text. It uses Gaussian mixture models (GMMs) and hidden Markov models that are iteratively updated to learn phoneme matches [McAuliffe et al., 2017]. Pre-trained English acoustic models are openly available. In this thesis, we use MFA to find exact phoneme locations within waveforms using ground truth or predicted transcriptions. Figure 2.5 visualizes the alignment of a waveform containing the spoken word "this".

```
name = "phones"
xmin = 0.04
xmax = 0.35
intervals [1]:
    xmin = 0.04
    xmax = 0.17
    text = "DH"
intervals [2]:
    xmin = 0.17
    xmax = 0.25
    text = "IH1"
intervals [3]:
    xmin = 0.25
    xmax = 0.35
    text = "S"
```

Figure 2.5: An example alignment of the spoken word "this" using MFA. The phonemes DH, IH1 and S and their temporal locations (in seconds) are listed in order of appearance.

2.7 Speaker Variation

Although there is a known set of phonemes, a given sequence of phonemes will hardly ever sound the same for different speakers. This hints towards another type of information encoded within a waveform or a spectrogram: speaker information. Voice characteristics like pitch, emotion, vowel duration and accents differ from person to person. This explains why sentences uttered by different speakers never sound identical. Speaker-specific information can be extracted in multiple ways. Originally, GMMs and Expectation-Maximization algorithms have been used for learning effective speaker representations [Kenny et al., 2005]. However, these methods were quickly replaced by deep neural methods. The most common way is to use features from

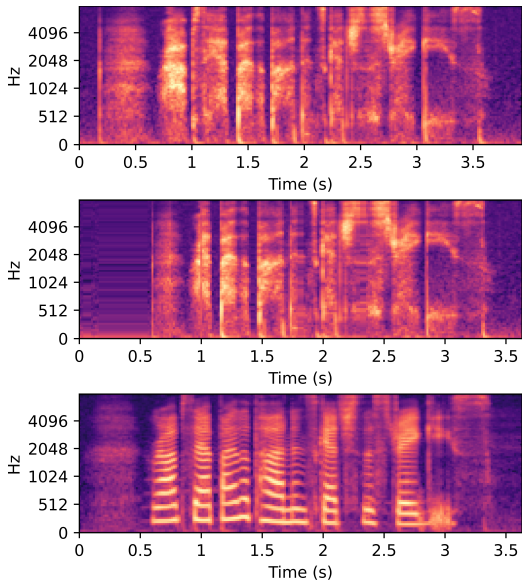


Figure 2.6: A whispered spectrogram (top), the whispered spectrogram warped to its spoken counterpart (middle) using DTW and the spoken spectrogram (bottom) containing the same utterance. Notice how the middle spectrogram now has the same width as the bottom spectrogram.

a bottleneck incorporated in a speaker classification network [Rouvier et al., 2015, Li and Wu, 2015, Jia et al., 2019]. A bottleneck is a fully-connected hidden layer with reduced size. The activations are a vectorial representation of a person's voice attributes. An arbitrary number of nodes can be used for capturing speaker embeddings. However, a small number—less than 20—would fail to cover all possible differences, and a size of 1024 or greater would be inefficient and disproportionate to the complexity of speaker attributes. It is hard to say what is actually encoded within each dimension because neurally learned features do not necessarily correspond to features that are interpretable by humans. Ultimately, a speaker embedding captures speaker characteristics that contrastively distinguish one speaker from another [Wang et al., 2017] and should not differ from utterance to utterance.

2.8 Streaming

Deep neural networks that synthesize waveforms usually do this asynchronously. That means that these network are often used to synthesize long chunks of audio in one go. However, Whispp aspires to create a *streaming* platform (such as Zoom or Skype) that can do real-time W2S conversion. For such an application to be useful, a long mouth-to-ear delay, or *latency*, is undesirable. Latency is measured in milliseconds (ms) and it is the time between audio being produced by the speaker and the listener hearing these sounds. In this work, we distinguish two types of latency: *structural latency* and *computational latency*. Structural latency is the minimum utterance length a model needs to produce an intelligible sample. Computational latency is how long it takes the model to synthesize a sample. Ideally, there would be no latency. However, there are operations between audio entering a system and emerging on the other side that inevitably take time. The G.114 recommendation issued by the International Telecommunication Union (ITU-T) defines an upper bound of 200ms because this hardly affects user experience negatively [Abbas et al., 1996].

Chapter 3

Related work

This chapter summarizes the relevant previous work related to this project. We will start by discussing the current state of neural speech processing, followed by an overview of both outdated and state of the art W2S models. The last section will discuss DPMs from their history to their current uses.

3.1 Vocoders

The importance of vocoders for speech processing can hardly be emphasized enough. Their task is to transform synthesized mel-spectrograms to waveforms, as they are able to induce the lost phase information. Since the introduction of the WaveNet [van den Oord et al., 2016] architecture, neural vocoders have shown superior results compared to traditional vocoders like Griffin-Lim [Griffin and Lim, 1983]. A problem with WaveNet is that it is *autoregressive*. In autoregressive models, audio samples are generated sequentially, where each value in a waveform is dependent on its predecessor. As we often need 16000 samples for 1 second of audio, inference is very slow. GAN [Goodfellow et al., 2014] based vocoders such as MelGAN [Kumar et al., 2019] and HiFiGAN [Kong et al., 2020] are *non-autoregressive*, meaning that they generate waveforms directly without explicit temporal dependence. These models are currently considered state of the art. The generators are trained to convert mel-spectrograms into waveforms, while the discriminators are trained to discriminate between generated and ground truth waveforms. Recently, diffusion models have also been used as vocoders [Chen et al., 2020, Kong et al., 2021, Koizumi et al., 2022]. These models take a noisy waveform as input and use the conditioning mel-spectrogram to gradually remove its noise and output a vocoded waveform. A favorable quality of these models is that they are also non-autoregressive, and do not require adversarial training, which can be unstable. Furthermore, they were shown to produce natural-sounding samples using as little as 6 reverse diffusion steps [Chen et al., 2020, Kong et al., 2021, Koizumi et al., 2022], measured through subjective listening tests. While most vocoders are only used for transforming synthesized mel-spectrograms to waveforms, a W2S vocoder that maps whispered spectrograms to spoken waveforms could also be theorized. We introduce methodology for training a W2S vocoder in Sections 3.4.5 and 4.1.

3.2 Voice Conversion

The objective of voice conversion (VC) is to make a speech sample generated by one speaker sound like it was generated by another speaker, while keeping the linguistic content intact. The most general and useful type of VC models is the one-shot many-to-many model. This type

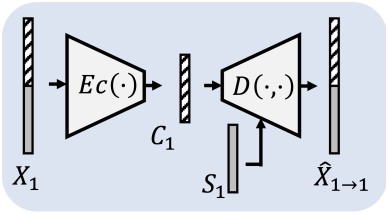


Figure 3.1: AutoVC, from [Qian et al., 2019].

of VC model only requires one reference utterance containing information of a target speaker (one-shot or zero-shot) to convert any source utterance to any target utterance (many-to-many), without retraining. An example zero-shot many-to-many VC model is AutoVC [Qian et al., 2019]. AutoVC is trained using a reconstruction loss similar to that of a Conditional Variational Autoencoder (CVAE) [Kingma and Welling, 2014, Kingma et al., 2014]. During training, the AutoVC model only has to reconstruct an utterance. I.e., the encoder $E_c(\cdot)$ extracts a latent representation of the content of an utterance. The decoder $D(\cdot,\cdot)$ reconstructs the utterance from the latent content representation, conditioned on the source speaker (Figure 3.1). During conversion, the decoder is conditioned on a target speaker and reconstructs the source utterance with the characteristics of the target speaker. Diffusion-based modelling has also been used to perform VC [Popov et al., 2021a]. Sections 3.4.5 and 4.2.1 explain the method used by Popov et al. [2021a] in more detail.

Moreover, VC models are used to translate from phonated speech to whispered speech. DNN-based VC models have especially been effective for generating whispered speech from phonated speech while remaining speaker-independent [Cotescu et al., 2020]. These speech-to-whisper (S2W) techniques are currently used in Amazon Alexa's Whisper Mode [Inc., 2018]. However, the reverse has never been attempted using a VC approach. A beneficial property of VC models for W2S conversion is that they can completely change the speaker characteristics in an utterance. This would allow a person with vocal chord problems to sound like any person he or she prefers if a speaker embedding for this target person is available.

3.3 Whisper to Speech

This section introduces the field of W2S, from its history to the state of the art. The most promising approaches and their weaknesses are outlined, followed by the baseline to which we compare our implementations.

3.3.1 Early Approaches

Before GANs, VAEs and other deep neural model architectures came into existence, methods such as Gaussian mixture Models (GMMs) were used to convert non-audible murmur (NAM) to speech [Toda and Shikano, 2005]. Even though the authors did not achieve perfect whisper to speech conversion, the approach was promising. Features of NAM and normal speech were later used instead of waveforms to train a GMM-based statistical model, MSpeC-Net [Malaviya et al., 2020], which uses autoencoders to find latent representations of NAM (not audible by the human ear), whisper and normal speech separately. A combination of loss functions then ensures the quality of these latent representations and provides a means of cross domain conversion. The NAM to whisper to speech route provided superior results. GANs have also been used to discover relations between modes of speech (DiscoGAN) [Kim et al., 2017]. An adaptation to DiscoGAN was used to learn relations between whisper and speech, and although the DiscoGAN had higher human ratings than a simple deep neural network (DNN), the results

were insufficient.

3.3.2 State of the Art

Generative neural networks remain the most commonly used architecture for W2S solutions to this day. In the state of the art whisper to speech model by Gao et al. [2021], self-attention modules have been added to GANs to help capture the informativeness within sentences, resulting in better intelligibility. A benefit of this method is that the paired whispered and phonated utterances do not have to be aligned, as the attention modules handles the informative pairing. However, the Root Mean Squared Error (RMSE) between the predicted pitch and the ground truth pitch is higher than previous GAN-based W2S models, yet Gao et al. [2021] report their attention-guided GAN to be superior to previous models in terms of subjective auditory perception.

3.3.3 Whispp's Approach

As mentioned earlier, GAN-based vocoders have been adapted for W2S. In these models, the generator is conditioned on a whispered utterance and its objective is to generate a spoken utterance. Whispp currently uses a GAN-based model that converts spectrograms to waveforms directly. As this model has highly competitive performance and stability, it will be used as a baseline throughout this thesis. Due to the competitiveness of the field of W2S, details of its architecture and training strategy will not be disclosed.

3.4 Diffusion-Based Modeling

In this section we present an overview of diffusion models, from their first use in image generation tasks to their current state of the art applications in vocoding and voice conversion. We will also cover the relation between score-based generative models and diffusion probabilistic models and how they can be seen as discretizations to stochastic differential equations (SDEs). (We do this because the generative models that are discussed later all have different theoretical foundations.)

3.4.1 Diffusion Probabilistic Models

Diffusion probabilistic models (DPMs) are inspired by non-equilibrium thermodynamics, a field in physics that is concerned with the seemingly random movement of particles that are not in equilibrium. The original idea of DPMs is to systematically and slowly destroy structure in a data distribution through an iterative forward diffusion process. We then try to learn a reverse diffusion process that restores structure in data, yielding a generative model that can be used to sample from the real data distribution while remaining flexible and tractable [Sohl-Dickstein et al., 2015].

We can model these forward and backward diffusion process using a Markov chain. Let $\mathbf{x}_0$ denote a sample from the distribution of interest ($q(\mathbf{x}_0)$) and let $\mathbf{y}$ denote a sample from a tractable prior distribution $p(\mathbf{y})$. The forward process of the Markov chain gradually adds different levels of noise $\beta_t \in (0, 1)$ to our unknown data distribution $q(\mathbf{x}_0)$ in T steps. We denote an intermediate sample that has been diffused with a certain amount of noise β_t by $\mathbf{x}_t$. The resulting distribution $p(\mathbf{x}_T)$ is analytically tractable because it is purely noise from a chosen prior (often an isotropic Gaussian distribution). The likelihood of the Markov chain is

defined as

$$q(\mathbf{x}_{0:T}) = q(\mathbf{x}_0) \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1}). \tag{3.1}$$

The conditional $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is the transition probability of $\mathbf{x}_{t-1}$ becoming $\mathbf{x}_t$ when $\mathcal{N}(\mathbf{0}, \beta_t\mathbf{I})$ is added. Because we will be using a Gaussian prior throughout this work, $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ can be written as a conditional Gaussian distribution:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}), \tag{3.2}$$

where $\mathbf{I}$ is the identity matrix.

The forward process $q(\mathbf{x}_{0:T})$ can be reparameterized so that we can sample a noisy datapoint $\mathbf{x}_t$ directly by adding a certain amount of noise to a data sample $\mathbf{x}_0$ once, instead of diffusing it iteratively with $\beta_{1:t}$. For this we first define $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$ and noise $\boldsymbol{\epsilon}_{t-1}, \boldsymbol{\epsilon}_{t-2}, \cdots \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. We can then write

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}_{t-1} && (3.3)\\
&= \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\bar{\boldsymbol{\epsilon}}_{t-2} && (3.4)\\
&= \dots && (3.5)\\
&= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon} && (3.6)
\end{aligned}$$

The resulting distribution of $\mathbf{x}_t$ becomes

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}). \tag{3.7}$$

An advantageous property of diffusion processes is that for sufficiently small noise levels β the reverse diffusion process has the same functional form as the forward process [Feller, 1949]. This implies that the reverse transition probability $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is also Gaussian. However, the mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\Sigma}$ of this distribution are unknown as we have no way of knowing what noise was added in a previous step. Consequently, $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are estimated with a neural network parameterized by $\boldsymbol{\theta}$. The following equations show the parameterization of the reverse diffusion process, where $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ denotes the approximation with $\boldsymbol{\theta}$ of the reverse transition probability $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$.

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) \tag{3.8}$$

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t), \boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)) \tag{3.9}$$

In equation 3.9, $\boldsymbol{\mu}_{\boldsymbol{\theta}}$ and $\boldsymbol{\Sigma}_{\boldsymbol{\theta}}$ are parameterized by a neural network. Using $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$, we are able to sample from the true posterior data distribution $q(\mathbf{x}_0|\mathbf{x}_T)$. Figure 3.2 visualizes the forward and reverse diffusion processes.

To train the neural network, we optimize the negative log-likelihood with the variational lower bound (ELBO):

$$\mathbb{E}[-\log p_{\boldsymbol{\theta}}(\mathbf{x}_0)] \leq \mathbb{E}_q\left[-\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] = \mathbb{E}_q\left[-\log p(\mathbf{x}_T) - \sum_{t\geq 1} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})}\right] = L. \tag{3.10}$$

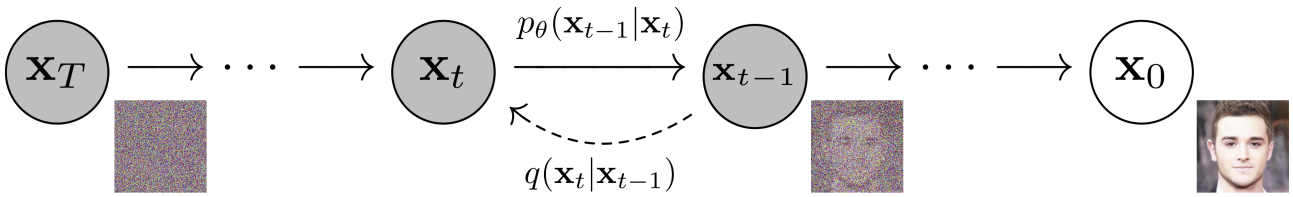


Figure 3.2: A visualization of the Markov chain underlying a DPM. Image from Ho et al. [2020].

For details on how this objective is found, refer to A.1. The objective is then split into terms that can be computed in closed form:

$$L_{VLB} = \mathbb{E}_q[\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1}^{T} \underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)}_{L_0}]. \tag{3.11}$$

Full derivation can be found in Appendix A.2. We note that the term L_T will be close to zero if $q(\mathbf{x}_T|\mathbf{x}_0)$ adequately resembles our prior distribution $\mathcal{N}(\mathbf{0}, \mathbf{I})$, so it can be ignored. Sohl-Dickstein et al. [2015] set the term $\log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)$ equal to the first step of the forward diffusion process (which is known) to avoid border effects. As this is the last step in the reverse diffusion process, optimizing for this final step as well can lead to unexpected behavior. The term L_0 can therefore be removed. The posterior

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}) \tag{3.12}$$

can be computed in closed form with parameters:

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t \tag{3.13}$$

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t. \tag{3.14}$$

This makes L_{t-1} computable, see Appendix A.2 for full derivations.

To parameterize $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ and $\boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ for approximate data distribution $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$, multiple approaches can be taken. Ho et al. [2020] found that predicting noise $\boldsymbol{\epsilon}$ in

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}(\mathbf{x}_t, t) \Big) \tag{3.15}$$

worked best compared to predicting $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ directly, especially when a reweighted form of L_{VLB} was used:

$$L_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \Big[\parallel \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \parallel^2 \Big]. \tag{3.16}$$

The covariance parameter is ignored during training as Ho et al. [2020] achieved their best results with $\boldsymbol{\Sigma}_{\boldsymbol{\theta}}$ set to σ_t^2, with $\sigma_t^2 = \beta_t$ or $\sigma_t^2 = \tilde{\beta}_t$. After training, a new sample $\mathbf{x}_{t-1}$ can be computed using Eq. 3.15:

$$\mathbf{x}_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}} \Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \Big)}_{\text{reparameterized } \boldsymbol{\mu}_{\boldsymbol{\theta}}} + \underbrace{\sigma_t \mathbf{z}}_{\text{fixed } \boldsymbol{\Sigma}_{\boldsymbol{\theta}}}, \tag{3.17}$$

with $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

It was shown that this procedure resembles Langevin dynamics and that the simplified variational lower bound (Eq. 3.16) resembles the objective used in score-based generative modeling. These subjects will be discussed in section 3.4.2 and 3.4.3 respectively.

3.4.2 Langevin Dynamics

In section 3.4.1 we mentioned that the equation used for sampling $\mathbf{x}_{t-1}$ from $\mathbf{x}_t$ using predicted noise $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ resembles Langevin dynamics (Eq. 3.17), where $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ can be seen as the gradient of the log-density $q(\mathbf{x})$. This fact is important as it allows us to draw a parallel between DPMs and score-based models. Langevin dynamics originate from non-equilibrium thermodynamics where they model Brownian Motion, i.e. the seemingly erratic movement of particles in a medium. After Albert Einstein first studied this motion statistically [Einstein, 1905], Paul Langevin formulated the first stochastic differential equation describing the random thermal fluctuations of the medium [Langevin, 1908]. The solution to this equation is a Markov chain [Uhlenbeck and Ornstein, 1930] that describes the movement of a particle from its starting point $q(\mathbf{x}_0)$ to an equilibrium $q(\mathbf{x}_T)$ where random movement is accounted for by adding Gaussian noise at each step, exacyly what is done by DPMs at inference. Welling and Teh [2011] have used Langevin dynamics to improve upon stochastic gradient optimization by adding a certain level of noise at each gradient step. It was shown that with a polynomial annealing schedule, the noise prevents overfitting, avoids getting trapped in local minima and enables sampling from the true posterior distribution. We will show in section 3.4.3 that the gradient of the log-density $q(\mathbf{x}_0)$ can be approximated using a *score* network that generates samples using Langevin dynamics, just like DPMs.

3.4.3 Score-Based Models

A generative principle similar to DPMs is score-based modeling. Score-based generative models (SGMs) belong to the same family as DPMs but rely on the *Stein score* [Liu et al., 2016] to represent probability distributions. The *score* is the gradient of the log-density function $q(\mathbf{x})$ at the input datapoint [Song and Ermon, 2020], pointing in the direction where the data density is the biggest. The score of a hypothetical 2-dimensional data distribution is visualized in figure 3.3. A useful property of the score function is that it is almost always tractable [Song and Ermon, 2020] due to the fact that the often intractable normalization constant of $q(\mathbf{x})$ vanishes in the gradient. A problem is, however, that if the data lies on a lower dimensional manifold, the score becomes undefined in the ambient space, i.e. the space where the log density is not supported. Another problem is that sampling will become inaccurate in regions where data is scarce. In these regions, the gradient is too weak to accurately follow to a higher density region. The solution to these problems is to perturb the data with various levels of Gaussian noise. This ensures that the data does not lie on a lower dimensional manifold. Also, the Gaussian noise will cause the low density regions of the data to contain more samples, thus increasing sampling accuracy. However, too much noise will make data inaccurate and too little noise causes the low density regions to not be explored enough. To account for this, a score network is trained to estimate the scores conditioned on noise level $\sigma_t, t \in 1, 2, \ldots, T$ from a schedule of decreasing noise levels $\sigma_1 < \sigma_2 < \cdots < \sigma_T$, analogous to $\{\beta_1 \cdots \beta_T\} \in (0, 1)$ in Section 3.4.1. Typically, T is in the hundreds or thousands. To generate samples, an *annealed* version of Langevin dynamics is used. The sample starts in a vector field of scores with the highest noise, and is gradually moved to a higher density data region while the noise decreases. This sampling strategy is inspired by simulated annealing [Song and Ermon, 2020, Kirkpatrick et al., 1983, Neal, 2001].

Score networks $\mathbf{s}_{\boldsymbol{\theta}}$ are trained with score matching [Hyvärinen, 2005] to approximate the score of the data density: $\nabla_{\mathbf{x}} \log q(\mathbf{x})$. The optimization objective for one score network is the Fisher divergence: $\frac{1}{2}\mathbb{E}[\| \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}) - \nabla_{\mathbf{x}} \log q(\mathbf{x}) \|_2^2]$, which can be simplified to

$$\mathbb{E}_{p(\mathbf{x})}\Big[\mathrm{tr}(\nabla_{\mathbf{x}} \mathbf{s}_{\boldsymbol{\theta}}) + \frac{1}{2} \| \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}) \|_2^2 \Big], \tag{3.18}$$

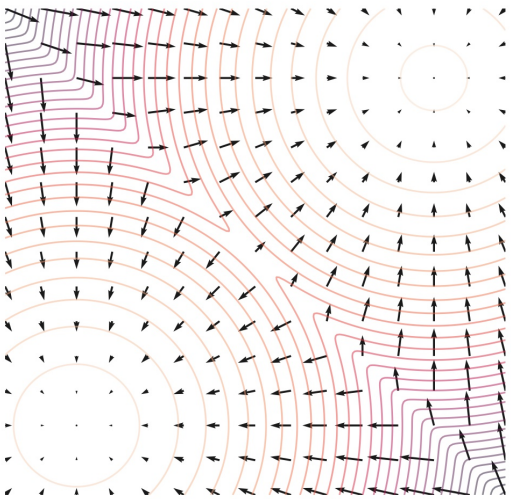


Figure 3.3: The (log-)gradient of a 2-dimensional data density is visualized here. The arrows point in the direction of the largest data density [Song, 2021].

with $\nabla_{\mathbf{x}}\mathbf{s}_{\boldsymbol{\theta}}$ being the Jacobian of $\mathbf{s}_{\boldsymbol{\theta}}$. However, due to $\text{tr}(\nabla_{\mathbf{x}}\mathbf{s}_{\boldsymbol{\theta}})$ not being scalable in computation for higher dimensions, the objective has to be adapted. The first possible solution is to perturb the data with noise distribution $p_\sigma(\tilde{\mathbf{x}}|\mathbf{x})$. The objective then becomes

$$\frac{1}{2}\mathbb{E}_{p_\sigma(\tilde{\mathbf{x}}|\mathbf{x})q(\mathbf{x})}[\parallel \mathbf{s}_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log p_\sigma(\tilde{\mathbf{x}}|\mathbf{x}) \parallel_2^2]. \tag{3.19}$$

This is called *denoising score matching* [Vincent, 2011] and can be a valid alternative. However, it is limited as it requires the noise to be very small so that $p(\mathbf{x}) \approx q(\mathbf{x})$. Another solution is *sliced score matching* [Song et al., 2019], which uses random vectors to approximate the trace of $\nabla_{\mathbf{x}}\mathbf{s}_{\boldsymbol{\theta}}$. These random vectors $\mathbf{v}$ are taken from a multivariate distribution $p_{\mathbf{v}}$ and do not perturb the actual data distribution $q(\mathbf{x})$. The objective for sliced score matching is defined as

$$\mathbb{E}_{q_{\mathbf{v}}}\mathbb{E}_{q(\mathbf{x})}\left[\mathbf{v}^{\mathbf{T}}\nabla_{\mathbf{x}}\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x})\mathbf{v} + \frac{1}{2} \parallel \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}) \parallel_2^2 \right]. \tag{3.20}$$

As mentioned previously, Langevin dynamics is used to generate samples with estimated score network $\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x})$. To generate a sample from log-density $q(\mathbf{x})$, we iteratively move a random sample $\tilde{\mathbf{x}}_T$ in the direction of the gradient of log-density $q(\mathbf{x})$:

$$\tilde{\mathbf{x}}_{t-1} = \tilde{\mathbf{x}}_t + \frac{\epsilon}{2}\nabla_{\mathbf{x}} \log q(\tilde{\mathbf{x}}_t) + \sqrt{\epsilon}\mathbf{z}_t. \tag{3.21}$$

Here, $\sqrt{\epsilon}\mathbf{z}_t$ with $\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the term that ensures that the model does not overfit on the lower dimensional manifold of the data distribution, as mentioned in the first paragraph of this section. We notice how the sampling scheme of score-based models (eq. 3.21) looks very similar to the reverse diffusion process of DPMs (eq. 3.17). We can argue that both generative model types belong to the same family and approximate the same score function under the hood [Song et al., 2021].

3.4.4 Stochastic Differential Equations

Before we start introducing Stochastic Differential equations, we note that in this section, $p_0(\mathbf{x}(0))$ is our unknown data distribution with $\mathbf{x}(0)$ being a sample from it with no noise added. A sample $\mathbf{x}_T \sim p_T(\mathbf{x}(T))$ is a sample from $p_0(\mathbf{x}(0))$ that is destroyed with noise level σ_T. As T is sufficiently large, $p_T(\mathbf{x}(T))$ becomes very close to a tractable prior distribution $\pi(\mathbf{x})$. This is different from Sections 3.4.1 - 3.4.3 where $p(\mathbf{x})$ was our tractable prior and $q(\mathbf{x})$

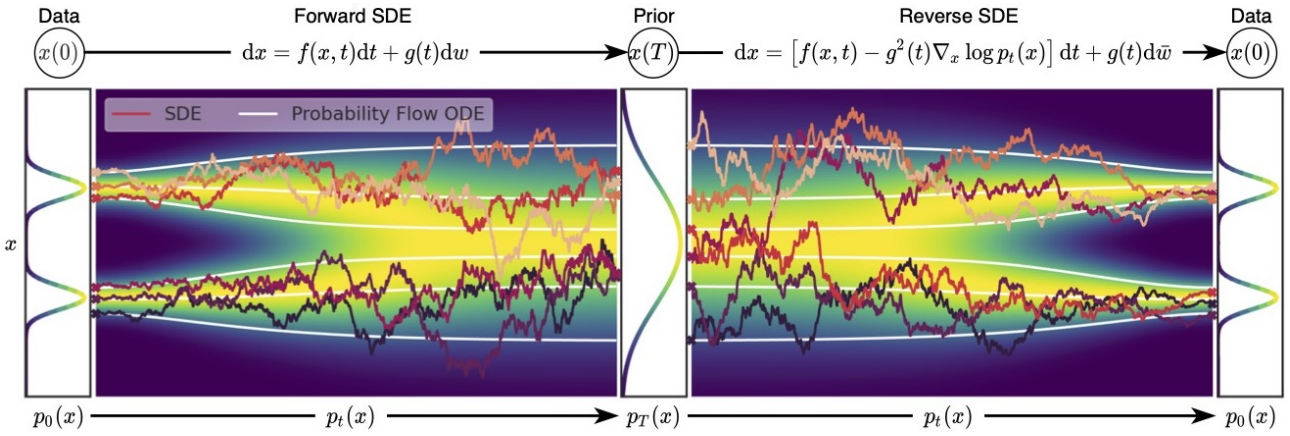


Figure 3.4: This figure shows the forward and backward SDEs. On the left side, we see data distribution $p_0(\mathbf{x})$, from which data points are taken as examples. The 6 reddish lines show the continuous process of infinitesimal Gaussian noise being added to the samples, eventually arriving at (a very close approximate of) an isotropic Gaussian prior $p_T(\mathbf{x})$. The right side of the figure shows 6 new samples of pure noise from our actual prior $\pi(\mathbf{x})$. The reverse SDE is then used to move the noisy samples back to data distribution $p_0(\mathbf{x})$ The curved white lines depict the probability flow ODE. Ordinary differential equations (ODEs) are derived from SDEs and allow for exact likelihood computation. ODEs will not be discussed further as they are irrelevant for our models. Image from [Song et al., 2021].

was used to denote the unknown data distribution. We use this new notation to be consistent with the literature and the figures used.

In Section 3.4.3 we discussed training multiple score-networks with different noise levels $\sigma_1 < \sigma_2 < \cdots < \sigma_T$ simultaneously to stabilize sampling. As $T \to \infty$, the sequential noising operations become a continuous stochastic process, which is a solution to a stochastic differential equation (SDE) of the form:

$$\mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t + g(t)\mathrm{d}\mathbf{w}, \tag{3.22}$$

where $\mathbf{w}$ is a Wiener process (i.e. Brownian motion), $g(t)$ is a diffusion coefficient and $\mathbf{f}(\mathbf{x}, t)$ is the drift coefficient of $\mathbf{x}(t)$ [Song, 2021]. The SDE in equation 3.22 transforms samples $\mathbf{x}(0) \sim p_0(\mathbf{x}(\mathbf{0}))$ continuously to samples $\mathbf{x}(T) \sim p_T(\mathbf{x}(\mathbf{T}))$ (very close to our prior distribution) by adding infinitesimal levels of noise. If this SDE could be reversed, we would have a continuous method of sampling $\mathbf{x}(0) \sim p_0(\mathbf{x})$ from our prior. Anderson [1982] discovered that the reverse of a diffusion process is also a diffusion process but one that runs backwards in time. This reverse-time SDE is defined as:

$$\mathrm{d}\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g(t)^2\nabla_{\mathbf{x}} \log p_t(\mathbf{x})]\mathrm{d}t + g(t)\mathrm{d}\bar{\mathbf{w}}, \tag{3.23}$$

where $\bar{\mathbf{w}}$ is a reverse Wiener process. Approximating $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ for all t enables us to sample $\mathbf{x}(0) \sim p_0(\mathbf{x})$. The complete SDE flow is visualized in Figure 3.4. Song et al. [2021] use denoising score matching in their work to estimate the noise conditioned score function $\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}, t)$ for the reverse SDE, similarly to how the discrete problem was solved in Section 3.4.3:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathbb{E}_t\Big\{\lambda(t)\mathbb{E}_{\mathbf{x}(0)}\mathbb{E}_{\mathbf{x}(t)|\mathbf{x}(0)}[\| \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log p_{0,t}(\mathbf{x}(t)|\mathbf{x}(0)) \|_2^2]\Big\}. \tag{3.24}$$

During training, t is sampled uniformly from the range $[0, T]$ and λ is a positive weighting function. The transition kernel $p_{0,t}(\mathbf{x}(t)|\mathbf{x}(0))$ from $\mathbf{x}(0)$ to $\mathbf{x}(t)$ is generally computable in closed form. Song et al. [2021] solve the reverse SDE with an Euler-Maruyama solver [Platen and Bruti-Liberati, 2010]. This solver is qualitatively similar to Langevin dynamics as it moves samples to high-density regions of $p_0(\mathbf{x}(0))$ by following the estimated score functions perturbed with Gaussian noise. When applied to our estimated reverse SDE, it discretizes the SDE using finite time steps and small Gaussian noise:

$$\Delta \mathbf{x} \leftarrow [\mathbf{f}(\mathbf{x}, t) - g^2(t)\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{t})]\Delta t + g(t)\sqrt{|\Delta t|}\mathbf{z}_t \tag{3.25}$$

$$\mathbf{x} \leftarrow \mathbf{x} + \Delta \mathbf{x} \tag{3.26}$$

$$t \leftarrow t + \Delta t \tag{3.27}$$

The formulation of diffusion-based models in terms of SDEs shows that DPMs and SGMs are both discretizations to a continuous diffusion process that allows us to sample from any timepoint t in the stochastic processes. Perturbing datapoint $x(0)$ with infinitesimal levels of Gaussian noise σ_t with $t \in [0, T]$ is a continuous way of destroying data with discrete levels $\mathcal{N}(\mathbf{0}, \sigma_1^2\mathbf{I}), \mathcal{N}(\mathbf{0}, \sigma_2^2\mathbf{I}), \ldots \mathcal{N}(\mathbf{0}, \sigma_T^2\mathbf{I})$ (SGMs) or with $\mathcal{N}(\mathbf{0}, \beta_1\mathbf{I}), \mathcal{N}(\mathbf{0}, \beta_2\mathbf{I}), \ldots \mathcal{N}(\mathbf{0}, \beta_T\mathbf{I})$ (DPMs) that allows for highly flexible sampling [Song, 2021]. SDEs should thus be seen as a continuous connection between DPMs and SGMs.

3.4.5 Applications

This sections moves on from mathematical theory and provides an overview of the results diffusion models achieved in literature.

Image Synthesis

Diffusion-based models have been originally applied in the field of image generation. [Sohl-Dickstein et al., 2015] showed that DPMs are tractable and efficient, while remaining powerful, flexible and exact. However, the diffusion process described in Section 3.4.1 requires the diffusion steps to be very small, which makes this approach not easily scalable. Moreover, due to the approach being likelihood-based, the models tend to generate blurry samples: likelihood-based models assign a high probability to training points, which does not prevent high probability being assigned to blurry points as well [Goodfellow et al., 2016]. To counteract these disadvantages, the likelihood-based objective was replaced by a *stein-score*-based [Liu et al., 2016] objective, discussed in Section 3.4.3. In SGMs, score matching [Hyvärinen, 2005] is used to learn the score of the logarithmic data density $\nabla_{\mathbf{x}} \log q(\mathbf{x})$, which is the gradient of the log-density function at the input data point. Annealed Langevin dynamics can then be used to produce samples. This score-based approach does not need adversarial training or Markov Chain Monte Carlo sampling, while generating results comparable to the best likelihood-based models and GANs. It was shown later that DPM's variational bound can be simplified to an objective that resembles denoising score matching [Ho et al., 2020]. This parametrization resulted in improved results that were state of the art, but not significantly better than existing generative methods. Furthermore, the results were achieved with CIFAR-10 [Krizhevsky, 2009] and LSUN [Yu et al., 2016], which are datasets with low diversity. Diffusion models were later improved by learning the covariance term of the source distributions $\boldsymbol{Sigma}$, which was first set to a constant (3.17). This resulted in an improved likelihood and also allowed to sample from diffusion models with fewer steps. It was also found that these improved DPMs achieved much better mode coverage than GANs according to their recall [Nichol and Dhariwal, 2021]. Numerous ablation studies and classifier guidance (a method for trading off diversity for fidelity using gradients from a classifier) already enabled DPMs to beat GANs [Dhariwal and Nichol, 2021]), but the reparameterization of diffusion models into SDEs caused diffusion models to become state of the art [Song et al., 2021].

Vocoders

Concurrently to the discoveries of the effectiveness of diffusion models in image generation, diffusion models were also applied in the field of audio synthesis as vocoders. WaveGrad [Chen et al., 2020] and DiffWave [Kong et al., 2021] are both neural denoising auto-encoding vocoders. WaveGrad is a SGM (discussed in Section 3.4.3) and DiffWave is a DPM (discussed in Section 3.4.1). Although the sensory experiences of viewing and listening are unalike, their forward and backward diffusion processes can be regarded as highly similar. In end-to-end vocoder architectures, where the input and output of a model are both waveforms, 1-dimensional Gaussian noise is added to a waveform in steps. The reverse diffusion process tries to approximate the noise and remove it so that the original waveform is reconstructed. Architectures that rely on mel-spectrograms use exactly the same procedure as described in Section 3.4.5. The quality of the novel diffusion-based vocoders inspired further research. For example, WaveGrad was adapted for Text-to-Speech (TTS) purposes, approaching performance of state of the art neural TTS models [Chen et al., 2021]. DiffWave was applied in the domain of audio denoising. The model was first trained as usual; after convergence, the part of the model used for learning spectrogram features was swapped for a larger model with the objective to learn the mapping of a degraded spectrogram to a denoised spectrogram [Zhang et al., 2021]. Subjective preference scores were used to show that the modified DiffWave model produced better results than the original DiffWave architecture. Although Chen et al. [2020] place WaveGrad in the field of both DPMs and SGMs, their training objective closely resembles the likelihood-based objective used in DPMs. However, there is one key difference. Where Ho et al. [2020] define the loss function as Eq. 3.28, the authors of WaveGrad redefine this loss to condition on a continuous noise schedule $\bar{\alpha}$ instead of discrete iteration index t, hereby offering improved flexibility and enabling the generation of high fidelity audio in as few as six refinement steps. Eq. 3.29 shows this reparameterization.

$$\mathbb{E}_{\mathbf{t},\mathbf{x}_0,\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon}-\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\sqrt{\bar{\alpha}_t}\mathbf{x}_0+\sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon},t\right)\right\|^2\right] \tag{3.28}$$

$$\mathbb{E}_{\bar{\alpha},\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon}_{\boldsymbol{\theta}}\left(\sqrt{\bar{\alpha}}y_0+\sqrt{1-\bar{\alpha}}\boldsymbol{\epsilon},x,\sqrt{\bar{\alpha}}\right)-\boldsymbol{\epsilon}\right\|_1\right] \tag{3.29}$$

Furthermore, the authors of DiffWave report vocoder performance with 20 reverse diffusion steps, conditioned on the ground truth spectrogram. The authors of WaveGrad use predicted mel-spectrograms, thus testing more realistically, as ground truth spectrograms are not available in practical applications. They also use as little as 6 reverse diffusion steps.

Apart from the quality of this novel range of vocoders, a great benefit of using diffusion for waveform synthesis is that DPMs are non-autoregressive. Previous autoregressive vocoders like WaveNet [van den Oord et al., 2016] and others [Mehri et al., 2017, Kalchbrenner et al., 2018] generate samples one by one, resulting in slow synthesis, especially because of high temporal resolution of waveforms and spectrograms. Non-autoregressive diffusion models do not have this issue, as the reverse diffusion process refines every value simultaneously.

Voice Conversion

The principle of diffusion has also been applied to VC (e.q. [Popov et al., 2021a]), where a novel SDE solver (Section 3.4.4) was introduced that allowed for faster synthesis while keeping sample quality high. The authors of this VC project consider the diffusion model to be a type of autoencoder. The forward diffusion process that adds Gaussian noise in steps can be seen as the encoder, while the reverse diffusion process acts as a decoder. A difference with

e.g. WaveGrad and DiffWave is that the prior distribution (i.e. the result of forward diffusion steps) is not pure Gaussian noise, but a low dimensional manifold of the data, a data-dependent prior. The authors consider this prior to be an *average* representation of a mel-spectrogram (on phoneme level). In the forward diffusion process, mel-spectrograms are mapped to average voice spectrograms, based on the average phonemes of the complete dataset. This average representation of a spectrogram is then transformed to a voice-converted spectrogram by the decoder (i.e. reverse diffusion).

Summary

This chapter discussed the work we consider relevant for W2S. We introduced current state of the art speech processing methods in Sections 3.1 and 3.2, followed by previous work in W2S conversion 3.3. Subsequently, we introduced diffusion models as a relatively new family of generative models with its roots in non-equilibrium thermodynamics and score-matching (Section 3.4). We concluded the mathematical theory by formulating DPMs and SGMs as discretizations to the same stochastic differential equation, thus bridging the gap between them. The end of Section 3.4 was used to give an overview of diffusion models in the field of speech processing. We explain how we use diffusion models for W2S in the next chapter.

Chapter 4

Method

In this chapter, we present our adaptation of diffusion models for whisper to speech conversion. Promising diffusion-based models have been designed specifically for the domain of speech. We redesign them for our problem of W2S conversion. There are two main types of solution to the W2S problem:

1. An end-to-end approach, that tries to learn a translation from a whispered spectrogram to a spoken waveform directly. We will call this approach the *spect-to-wave* approach from now on.
2. An approach that tries to learn a translation from a whispered spectrogram to a spoken spectrogram. This spectrogram is then transformed into a waveform using a trained vocoder. We will call this approach the *spect-to-spect* approach from now on.

The spect-to-wave approach would be beneficial as a vocoder would then be unnecessary. This is an advantage because converting a spectrogram to a waveform separately is inefficient. On the other hand, it has been found that end-to-end approaches cannot compete with the spect-to-spect approach in text-to-speech diffusion tasks [Popov et al., 2021b]. Although this does lower expectations for end-to-end approaches, it does not rule them out completely. WaveGrad [Chen et al., 2020, 2021], DiffWave [Kong et al., 2021, Zhang et al., 2021] and diffusion-based voice conversion [Popov et al., 2021a] (DiffVC) all showed excellent results as diffusion-based vocoders, denoisers or voice converters. As these projects are unparalleled in their respective fields, the models should provide the highest chance of success in solving the W2S problem.

As a final contribution, we explore how well our best diffusion model performs in a streaming setting. To our knowledge, diffusion models have never been adapted for streaming purposes. We will test the limits of diffusion models out of the box and explain what can be done to further help streaming performance in Section 7.1.

4.1 Spectrogram to Waveform

In this section, we present the two models we use to implement the end-to-end approach to W2S, that of transforming whispered spectrograms to voiced waveforms.

4.1.1 WaveGrad

WaveGrad [Chen et al., 2020] was originally proposed for spect-to-wave synthesis, with pairs of voiced spectrograms and voiced waveforms being used as data for training. To adapt WaveGrad

for W2S, the ground truth voiced mel-spectrogram will be exchanged for a ground truth whispered mel-spectrogram. To temporally match the ground truth whispered mel-spectrograms with the voiced waveforms, we use dynamic time warping (DTW). The original model was trained on samples with a sample rate of 24kHz. However, as our baseline model was trained with samples that have a sample rate of 16kHz, the WaveGrad W2S model will be trained on samples with 16kHz. The model architecture is kept the same as the WaveGrad Base model consisting of 4 downsampling blocks, 5 upsampling blocks and 5 feature-wise linear modulation (FiLM) blocks in between. Downsampling blocks and upsampling blocks are combinations of convolutional layers with activations in between. Downsampling blocks downsample the temporal dimension of the noisy waveform and upsampling blocks increase the dimensions again to the dimensions of the input waveform. A graphical depiction of the downsampling and upsampling blocks is given in Figure 4.1. The FiLM block, first used by Dumoulin et al. [2018], learns scaling (γ) and shift (ξ) vectors that are used to combine information from the noisy waveform and the input mel-spectrogram with this formula:

$$\gamma(D, \sqrt{\bar{\alpha}}) \odot U + \xi(D, \sqrt{\bar{\alpha}}), \tag{4.1}$$

with $\odot$ being the hadamard product. Figure 4.2b shows the architecture of a FiLM block. With all blocks combined, the total model size is 15M parameters. The network architecture is shown in Figure 4.2a. Since WaveGrad is very good at mapping spoken spectrograms to spoken waveforms we find it valuable to explore how well WaveGrad can convert whispered spectrograms to spoken spectrograms. Due to the differences between whispered and voiced speech explained in Section 2.5, we expect the adapted WaveGrad model to have more difficulty synthesizing spoken data from whispered data. The pitch and formant frequencies have to be synthesized by the model with only the whispered spectrogram as a reference. Even though the result may not be perfect, it will still be worth seeing how a diffusion-based vocoder works on whispered data without specific architectural changes.

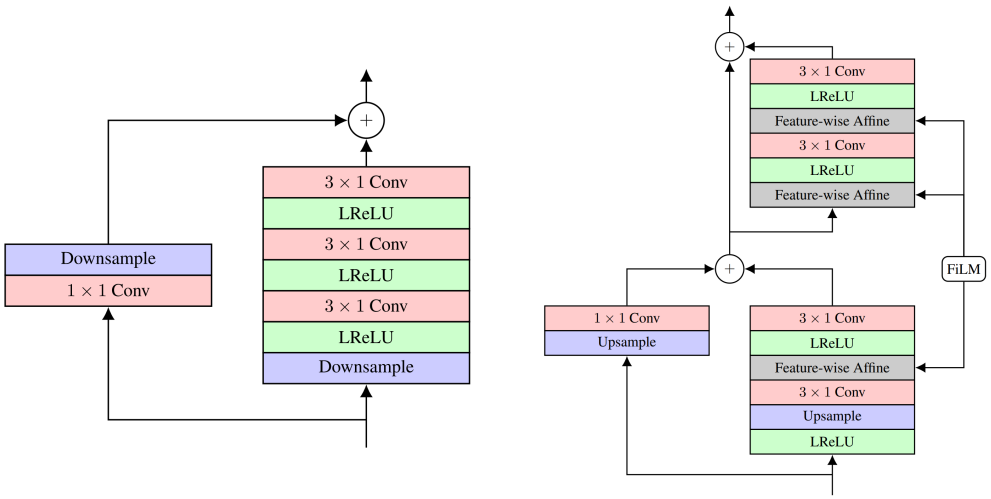
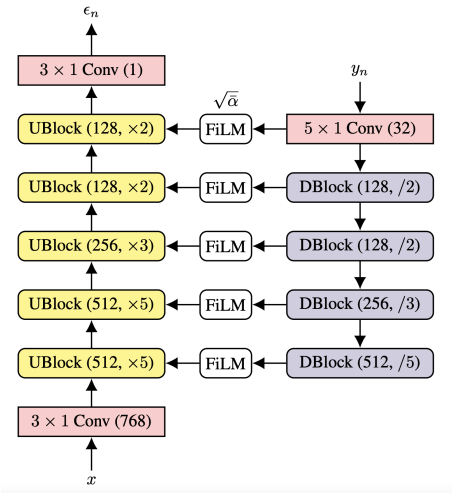


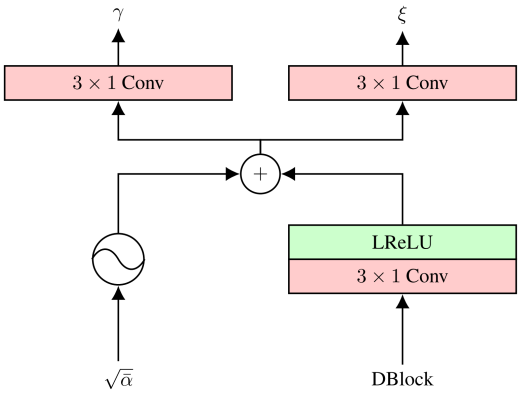
Figure 4.1: A downsampling block (left) and an upsampling block (right) as used in the architecture of WaveGrad. Figures from [Chen et al., 2020].

4.1.2 DiffWave

DiffWave [Kong et al., 2021] is similar to WaveGrad, but it has also been applied to atypical speech. We take inspiration from Zhang et al. [2021], who have successfully adapted DiffWave to restore degraded speech. In their work, a vocoder is modified to enhance speech quality



(a) The original WaveGrad network architecture. Here y_n is the model input at iteration n and x is the conditioning mel-spectrogram. The model predicts noise ϵ_n given y_n, x and continuous noise schedule $\sqrt{\bar{\alpha}}$. Figure from [Chen et al., 2020].



(b) A Feature-wise Linear Modulation (FiLM) block as used in the architecture of WaveGrad. Figure from [Chen et al., 2020].

after speech samples are degraded using clipping or compression. The original DiffWave model upsamples the conditioning mel-spectrogram to match the dimensions of the noisy waveform with a series of simple transposed 2D convolutions. To adapt the model for degraded speech, a DiffWave model is first trained to convergence. Afterwards, the mel-spectrogram upsampler is replaced with a deep CNN upsampler, which is trained to alter the degraded mel-spectrogram to match that of the original speech. The model is trained using the original speech waveform, but conditioned on the degraded speech mel-spectrogram. Post-training, only the degraded mel-spectrogram is used as input and the model generates an estimate of the original speech. The architecture of the original DiffWave model and a visualization of the adaptation Zhang et al. [2021] made are shown in Figure 4.3. For our use case, the degraded mel-spectrograms will be exchanged for whispered mel-spectrograms, again aligned with their spoken counterparts using DTW. The objective of the deep CNN upsampler then becomes to learn a mapping between the whispered mel-spectrogram and the voiced mel-spectrogram. This is challenging as pitch information is not absent in a degraded spectrogram like it is in the whispered spectrogram. This gives the original model an advantage over the the model adapted for W2S. However, we believe the model performs well enough to still be worth adapting for W2S. The authors of DiffWave experimented with optimal noise schedules. Two of these schedules will be used and compared, one with 50 diffusion steps and one that has 6. Details will be discussed in Section 5.2.

4.2 Spectrogram to Spectrogram

4.2.1 DiffVC

To implement the spect-to-spect approach, we adapt DiffVC Popov et al. [2021a] for W2S. This model is able to successfully translate a mel-spectrogram uttered by any person to a mel-spectrogram uttered by any other person. If we consider a hypothetical speaker that only produces *whispered* voice utterances, it is worth investigating if the conversion to an arbitrary spoken voice can be made. As mentioned in Section 3.4.5, source spectrograms are translated to average mel-spectrograms by the encoder first. Then, the decoder converts this average mel-spectrogram to a voice-converted spectrogram. The architecture used by Popov et al. [2021a]

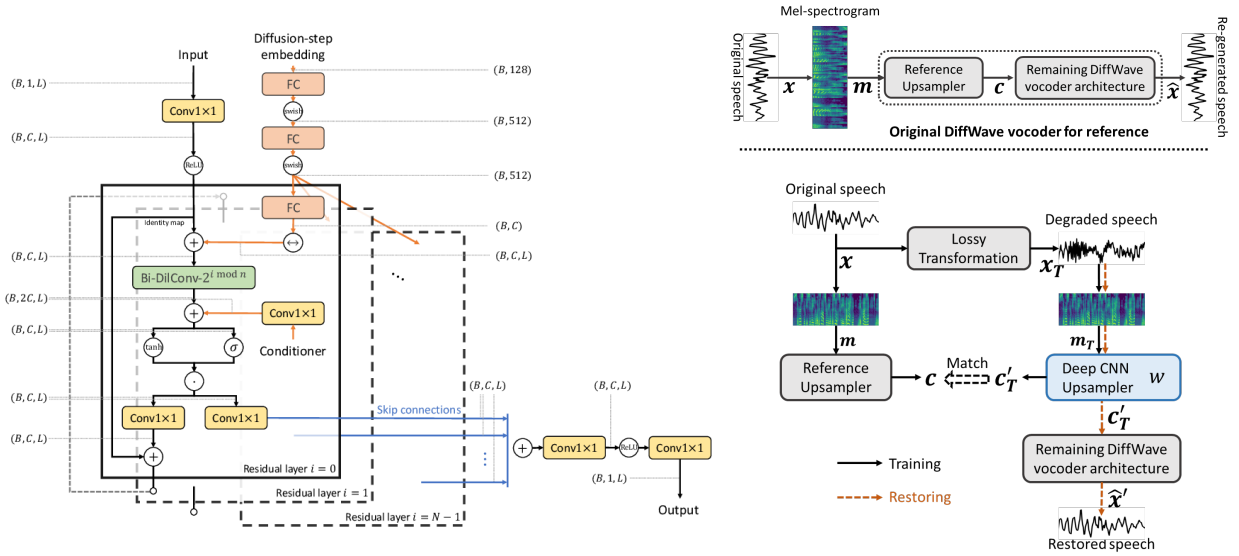


Figure 4.3: The original DiffWave architecture (left) and the modified training workflow (right). Figures from [Kong et al., 2021, Zhang et al., 2021] respectively.

is visualized in Figure 4.4.

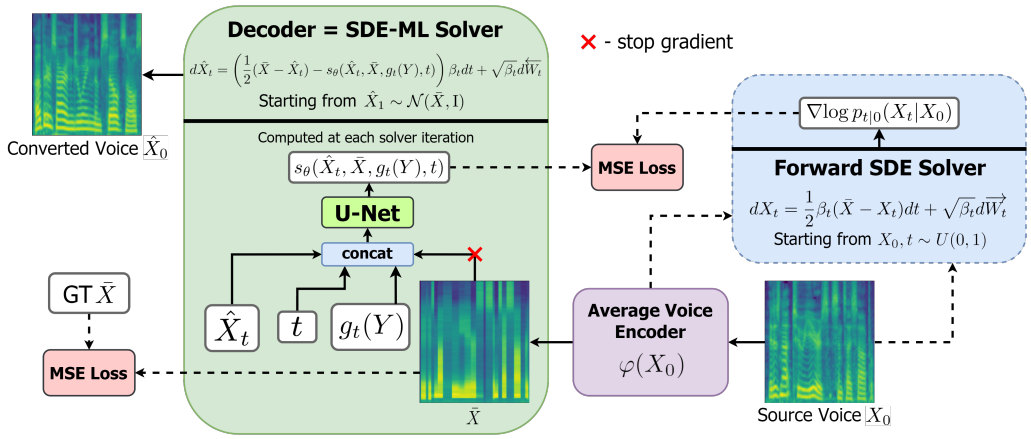


Figure 4.4: The model architecture for DiffVC. The encoder is trained to map whispered spectrograms to phonated average voice spectrograms. The average spoken spectrograms are then converted back to spoken spectrograms by the decoder, conditioned on timestep t and target speaker information ($g_t(Y)$) consisting of a noisy target spectrogram Y_t and the speaker embedding generated from Y. The noisy source spectrogram $\hat{X}_t$ is generated by diffusing source spectrogram X_0 towards average voice spectrogram $\bar{X}$. Figure from [Popov et al., 2021a].

Encoder

To train the encoder, ground truth average representations are generated as follows. MFA is used to find phoneme locations in both voiced and whispered utterances. The average voice spectrogram is constructed by iterating over the whispered spectrogram column-wise an replacing every column with a column containing the spoken average of the phoneme in that column. To prevent bias, only the training part of the dataset is used to calculate average phonemes. Figure 4.5 shows a whispered spectrogram and the corresponding ground truth average spectrogram. The encoder is then optimized to convert a whispered spectrogram to an average spoken spectrogram.

We additionally explore a whisper to gridphoneme (W2P) prediction model as an alternative for the encoder. A gridphoneme sequence is a sequence of per-frame phoneme predictions.

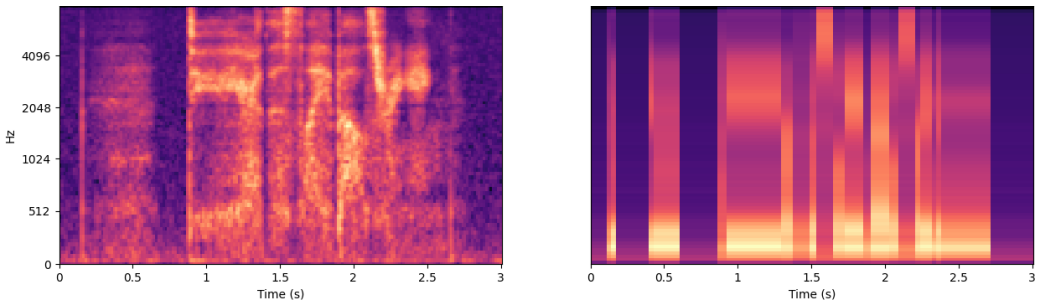


Figure 4.5: A whispered spectrogram (left) and an average spectrogram (right).

Each frame (column) in a mel-spectrogram is assumed to correspond to one phoneme. W2P models predict the gridphoneme sequence present in a whispered utterance. These predicted gridphonemes can then be used in combination with the average voiced phonemes to construct the average phoneme spectrogram. A W2P model is provided by Whispp. This model will not be discussed in more detail because it has to remain confidential. The W2P model has a phoneme error rate of approximately 20%, which is suboptimal, but it can still be used as a proof of concept. A possible benefit of W2P models is that the phoneme predictions could be refined with a phoneme language model. However, we consider this out of scope for our work. For this experiment, we train the DiffVC decoder with ground truth average voice spectrograms instead of those produced by the encoder. This is done to prevent bias towards average spectrograms generated by an encoder. We include the performance of this ground truth decoder in the results as a reference.

Decoder

In the original training setup of the DiffVC decoder, the model takes as input a spoken spectrogram, a speaker embedding derived from this spoken utterance and a random timestep t. The spoken spectrogram is split into two separate spectrograms. The first spectrogram is the main training datapoint X_0 and the second spectrogram is used as a reference that contains additional information the speaker embedding did not capture. The decoder is trained to estimate the noise level given timestep t, which is used to diffuse both spectrograms towards their respective average representations.

Two routes can be taken to train the decoder, a S2S route and a W2S route. For our S2S route, we must assume that the encoder generalizes well to spoken utterances, as both the source spectrogram and the target spectrogram will be voiced spectrograms and the encoder is trained to map whispered spectrograms to spoken average representations. The advantage is that no alignment is required. However, it is possible that the decoder will not generalize well to whispered utterances after taking this route, because it has never seen average representations generated from a whispered utterance during training.

For the W2S route, whispered source spectrograms are added to the training procedure. The whispered spectrograms are aligned with their spoken counterparts using DTW and then they are used to generate the average voice representations using the trained encoder. A voiced spectrogram is used as X_0, just like in the S2S route, only now we diffuse towards an average representation generated from a whispered spectrogram. A possible downside is that DTW has a negative effect on training. We follow both approaches and compare their performance.

4.3 Streaming

For optimal end-user experience, a W2S conversion system should have a low latency. More precisely, latency should be kept below 200ms. A latency bigger than 200ms would cause noticeable delay. However, the time it takes for an audio sample to reach the model, and for it to then be sent to the receiver often sums up to 100ms already, which leaves 100ms as actual model computation time. As vocalizing a sentence always takes more than 100ms, W2S conversion should produce output incrementally while a user is speaking. To achieve this, we send data to the model in chunks of X ms (as visualized in Figure 4.6). Still, there are

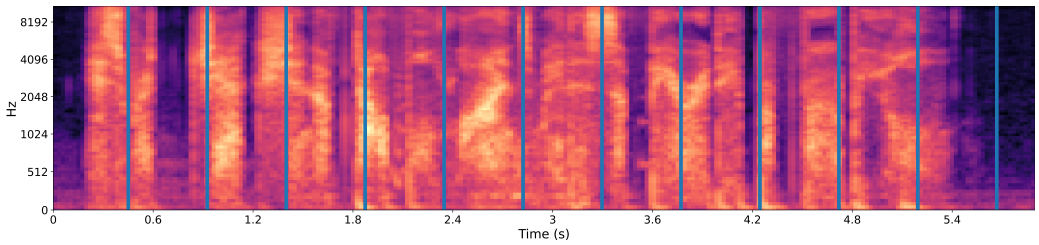


Figure 4.6: A whispered spectrogram that has been split into chunks with an arbitrary chunk length of 20 frames. The blue lines show where the spectrogram is split.

multiple problems with this approach. On the one hand, neural audio synthesis models such as vocoders generally rely on convolutions for their optimized learning. Convolution kernels need zero-padding to synthesize a complete utterance. Without padding, the convolution operation would cause small parts at the start and end of an utterance to disappear. When audio is split in chunks and concatenated after conversion, the padding on both sides results in noticeable border artifacts which negatively affect the user experience. On the other hand, state of the art speech synthesis models often make use of attention models for distinguishing the important parts in an input spectrogram [Gao et al., 2021, Popov et al., 2021a]. This means that we cannot make the size of the input chunks too small, as a receptive field of models with attention modules is much bigger.

To combat the first problem, we introduce an overlap between chunks so that missing information can be filled in. Two approaches can be taken to concatenate the converted output to the final result. The naive strategy is to construct the overlap so that the end of the first chunk is overlapped by a second chunk. The converted second chunk is then pasted over the first chunk at the start of the overlap. This strategy prevents border artifacts at the end of subsequent chunks, but border artifacts can still appear at the start of a chunk. A more sophisticated strategy is to create overlap on *both* sides of a chunk. The middle part of a converted chunk is concatenated to the final result. In the process, the parts where border artifacts could appear are sliced off. Figure 4.7 visualized the two approaches. We will explore how small the chunk size can be set without decreasing model performance excessively. We will also vary overlap size and report what effect it has on sound quality.

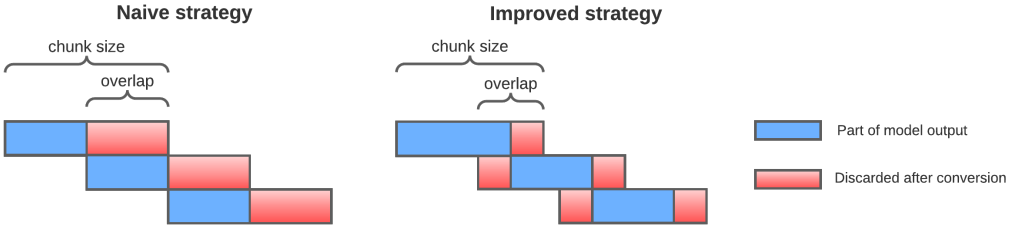


Figure 4.7: Two overlap strategies. Using the left strategy, border artifacts could still appear at the start of the blue parts. Using the right strategy, border artifacts are prevented at both sides.

The second problem is harder to solve as the attention modules prevent us from reducing the receptive field size too much. This is because attention modules inherently require more information than plain convolutional layers. We could alleviate this by masking out all the attention weights of forward-looking connections during training the attention module. However, this would inevitably have a negative effect on model performance, and we consider this research direction outside the scope of our experiments.

Furthermore, the inference schedule requires an important adaptation. As discussed in Section 3.4, Langevin dynamics is used for sampling, to prevent collapse into a lower dimensional manifold. The small amount of noise that is added at each inference step could result in chunks that sound differently, mostly in terms of loudness. To circumvent this, we assume that the reverse diffusion process has been learned robustly enough so that we can omit the noise from Langevin Dynamics at each inference step. We will document what effect this has on inference in Section 6.3.

In the next chapter, we introduce the dataset used for training the models. We propose an experimental setup to compare the models' performance against the Whispp baseline. We also explain how we measure streaming performance.

Chapter 5

Experiments

We train two different spect-to-wave models and one spect-to-spect model for W2S. We test them in a standard and in a streaming setting. For the standard setting, we test model intelligibility using word error rate (WER), phoneme accuracy (PA), phoneme error rate (PER) and character error rate (CER). We further test intelligibility and measure naturalness using a subjective listening test. We test the streaming performance of our models by measuring how much we can reduce latency without adding streaming artifacts in the synthesized speech. We will evaluate the gathered metrics and use them to make the following contributions: (i) we conclude whether diffusion models can be used for W2S conversion, and how effective they are compared to the baseline used by Whispp; (ii) we prove which one of the spect-to-wave or spect-to-spect approaches is the better approach; (iii) we are the first to use diffusion models for streaming and conclude how well these models work in a streaming setting without changing the model architecture.

5.1 Dataset

The dataset that will be used to train all models is the whispered TIMIT (wTIMIT) corpus [Lim, 2011]. The wTIMIT corpus consists of approximately 18 hours of paired whispered and voiced speech samples with a sample rate of 44.1kHz. The dataset contains 450 sentences that were all whispered and spoken by 47 speakers, of which 25 are male and 22 are female. 19 speakers have a Singaporean-English accent and 28 speakers have a North American accent. To accurately present our data distribution, two speakers from both accent groups are held out from training to be used for testing, which gives us a train-test split of approximately 90/10. For the DiffVC model, 30 utterances per test speaker are held out to be used as reference spectrograms and for the generation of speaker embeddings (see Section 2.7). We downsample the audio files to 16kHz in preprocessing for efficiency and consistency with the baseline. Spectrograms are created with 80 mel-frequencies and a Hamming window of size 1024 with a hop size of 256. For a sample rate of 16kHz, this translates to windows being 64ms long and 16ms apart.

Whispp internally uses a proprietary dataset. This also is a paired whispered and voiced speech corpus with the same technical properties and preprocessing steps as wTIMIT. However, it contains more speakers, more sentences and was recorded with better quality equipment. The Whispp dataset also contains other languages besides English, but we will not be using those in this work. Table 5.1 shows the main data statistics. The Whispp dataset will be used next to wTIMIT for comparison against the baseline model introduced earlier, in Section 3.3.3. The baseline Whispp uses showed great improvements when trained using their proprietary dataset. We also test our wTIMIT-trained models on the Whispp dataset to see how well they

generalize.

	wTIMIT	Whispp (EN-US)
Nr. Speakers	47	118
Avg. Nr. utterances per speaker	438.4	398.6
Avg. whispered utterance length (s)	3.38	3.51
Avg. spoken utterance length (s)	3.03	3.48
Total duration of whispered audio (h)	19.52	45.86
Total duration of spoken audio (h)	17.31	45.59
Total paired utterances	21047	47031
Languages	EN-US, EN-SG	EN-US

Table 5.1: The main properties of wTIMIT and the English portion of the Whispp dataset.

5.2 Hyperparameters

For WaveGrad and DiffWave, we use a batch size of 128, the maximum allowed by our infrastructure as larger batch sizes were shown to lead to better performance Chen et al. [2020]. For the deep upsampler trained to restore whispered spectrograms in DiffWave, we select a batch size of 8. This is again the largest possible batch size. We use a learning rate of 2×10^{-4} for both WaveGrad and DiffWave. Chen et al. [2020] showed that WaveGrad performed the best using a linear noise schedule that increases from 1×10^{-4} to 5×10^{-3} in 1000 steps. We will use this noise schedule as well during training and inference. For DiffWave we use a noise schedule that increases from 1×10^{-4} to 5×10^{-2} in 50 steps, both during training and at inference time. A fortunate feature of DiffWave is that we do not have to retrain the network when performing inference with different noise schedules. We use the smallest noise schedule proposed for DiffWave with hand picked values $\{0.0001, 0.001, 0.01, 0.05, 0.2, 0.5\}$.

DiffVC-based models are trained with a learning rate of 5×10^{-4} for the encoders and 1×10^{-4} for the decoders, following Popov et al. [2021a]. We use a batch size of 64 for the encoder and 8 for the decoder, which are the maximum sizes allowed by our GPU memory. A noise schedule is not needed for DiffVC because it is based on SDEs. For inference, we use 6 denoising steps to generate results in both standard and streaming settings. This is the smallest amount of steps tested by Popov et al. [2021a], who showed that samples generated with 6 and 30 steps often did not differ in terms of subjective quality. We will train the encoders for 500 epochs which takes approximately 2 days. The decoders will be trained for 110 epochs, following Popov et al. [2021a], taking approximately 6 days. The last checkpoints are used in our experiments.

5.3 Model Evaluation

To measure model performance, we use a combination of objective and subjective performance metrics, combined with a qualitative analysis of language-specific features. All metrics are computed over the held-out test set.

5.3.1 Objective Evaluation

Two simple objective metrics that measure the intelligibility of synthesized speech are word error rate (WER) and character error rate (CER). WER and CER are measured between a ground truth utterance transcription and the transcription of a synthesized waveform obtained

using an automatic speech recognition (ASR) model. In our experiments, we use QuartzNet [Kriman et al., 2020], a competitive open-source ASR model from the NeMo toolkit [Kuchaiev et al., 2019] created by NVIDIA. QuartzNet was not trained to generalize well on whispered data, so to benchmark it on whispered data would result in biased error rates. We will benchmark QuartzNet on voiced data to obtain ground truth error measurements.

WER and CER are calculated by summing all substitutions (words/characters altered by model), insertions (words/characters added by model) and deletions (words/characters removed by model) and dividing by the length of the ground truth transcription. This formula is based on the Levenshtein distance [Levenshtein et al., 1966]. A problem with WER and CER is that they use an ASR model. This may introduce transcription errors which alter measurements. Our measurements of WER in particular will suffer from the limitations of the underlying ASR model, QuartzNet, which does not take word/character context into account. Homophones, e.g. "flower" and "flour", will not be distinguished by QuartzNet based on the surrounding words. Consequently, some word and character errors will be due to poor ASR model performance instead of poor W2S model performance. Because phoneme-specific error is not affected by this problem as much as WER and CER, we also measure phoneme accuracy (PA) and phoneme error rate (PER). PA is the amount of error between per-frame gridphoneme sequences. These have the same length so we can easily count substitutions. PER is measured as Levenshtein distance between the collapsed phoneme sequences. Although PA and PER are very similar to WER and CER in computation, we cannot use ground truth transcriptions directly as they do not contain phonemes. We obtain the ground truth gridphoneme sequence by aligning the ground truth *whispered* samples with MFA (see Section 2.6.2). Predicted transcriptions are used to obtain phoneme sequences for synthetic samples. Whispered ground truth samples are used for two reasons: (i) the length of a synthetic spoken sample and a whispered sample cannot differ, enabling us to correctly measure PA; (ii) pronunciation can differ between whispered and voiced speech. We expect W2S models to perfectly preserve pronunciation in the *whispered* utterance, so measuring error between synthetic voiced utterances and ground truth voiced utterances would make PA and PER of 0% theoretically impossible. PA and PER allow us to investigate how well our models transfer the phonetic content of a whispered utterance to a voiced utterance. We extend our evaluation of the phonetic error between a whispered sample and a synthetic voiced sample by gathering all phoneme substitutions, as detected in our PA calculations. This will give us an insight into which phonemes are often mistaken for others.

5.3.2 Subjective Evaluation

An important feature of synthesized speech is whether it sounds natural. As naturalness is a subjective feature that cannot easily be quantified, we decide to use a human evaluation test to assess this feature. We also obtain intelligibility ratings in this test as we want to see how they compare against our objective metrics. For this human evaluation, we choose our three best models to compare them against the Whispp baseline.

Synthetic speech quality is often assessed using Mean Opinion Score (MOS) tests. MOS tests generally take the form of a survey that asks participants to score audio samples with an integer ranging from 1 to 5, where 1 corresponds to lowest and 5 corresponds to the highest perceived quality. Although MOS is a widely accepted subjective metric, it has some noteworthy drawbacks. For example, the mood of a subject tends to have a large influence on the generated scores [Zieliński, 2006] which is undesirable. Furthermore, there often is high variability between scores given by different participants [Zieliński, 2006], which can make our final scores hard to interpret. We designed a test that is able to compare all four models at

once, while mitigating the problems MOS tests are often subject to. Our test is inspired by the preference score or ABX test often used in voice conversion [Sisman et al., 2020], where 10 sets of 4 randomly selected synthetic test samples produced by each model had to be ranked from best (rank 1) to worst (rank 4). The ground truth spoken sample was provided for reference, as well as the transcription as some samples proved difficult to understand and we did not want this to interfere with the ranking. We also added a warm-up set containing one very good sample and one very bad sample in terms of both intelligibility and naturalness. This was an opportunity for participants to learn how to rank samples, and it was also used to filter bad submissions. Submissions completed in less than 10 minutes were also discarded as through internal evaluation we concluded that the test requires at least 10 minutes to be completed faithfully.

As our rating scale is fixed, meaning that ranks 1 through 4 all have to be used, we do not have the problem of the mood of participants interfering with the ratings (annotator bias). The final score R_m for a model is calculated as the sum of the ranks for all samples produced by the model divided by the number of sample sets and by the number of participants:

$$R_m = \frac{\sum_{i=1}^{N} \sum_{s=1}^{S} r_{ims}}{N \cdot S}, \tag{5.1}$$

with N denoting the number of participants and S denoting the number of sample sets. Because the averaging over ratings can hide informative characteristics of the rating distribution, we also provide an in-depth analysis of the rankings gathered with the listening test to prevent biased conclusions. To measure agreement between participants, we employ Kendall's W [Dodge et al., 2006]. This will give us an estimate of the difficulty, clarity, and subjectivity of the task. Kendall's W with n participants and k sets of rankings is defined as follows:

$$W = \frac{Q}{n(k-1)}, \tag{5.2}$$

where Q is the Friedman test statistic [Friedman, 1937] resulting from the following equations:

$$\bar{r}.j = \frac{1}{n} \sum_{i=1}^{n} r_{ij} \tag{5.3}$$

$$Q = \frac{12n}{k(k+1)} \sum_{j=1}^{k} (\bar{r}.j - \frac{k+1}{2})^2, \tag{5.4}$$

where r_{ij} is the rank of model x_{ij}, ranked by participant i in set j. Our interpretation of Kendall's W follows Table 5.2. This partitioning of agreement often differs in literature, and we should note that its thresholds are arbitrarily chosen. The thresholds in Table 5.2 are from [Moslem et al., 2019].

W	**Interpretation**
0	No agreement
> 0.1	Weak agreement
> 0.3	Moderate agreement
> 0.6	Strong agreement
1	Perfect agreement

Table 5.2: Interpretations of Kendall's W

5.4 Streaming

There are multiple objective and subjective qualities that define a good streaming model. Firstly, latency should be as low as possible. Secondly, there cannot be border artifacts. Thirdly, Real Time Factor (RTF), the ratio of computation time and chunk length, should be less than 1.

We define two types of latency: computational latency and structural latency. We measure computational latency as the time it takes to convert a whispered chunk to a voiced chunk and structural latency as the smallest chunk size that can be converted successfully without structure related problems (e.g. attention modules failing). The total latency is the sum of computational latency and structural latency. The presence of noticeable border artifacts will be evaluated subjectively by us. We do not need more participants for this as we can easily decide from listening to samples whether there are streaming artifacts present in a sample. For a W2S task, RTF is the time it takes to convert a whispered chunk a to a phonated chunk b divided by the duration of chunk a:

$$\text{RTF} = \frac{time(model(a))}{chunk_length(a)}. \tag{5.5}$$

When the average RTF ≤ 1, it takes the same or less time to convert a sample compared to producing it. Obviously RTF should be as low as possible, but RTF ≤ 1 is indispensable in streaming settings. As we need multiple forward passes through the model when using multiple chunks, we report the average RTF over all chunks passed through the model.

Chapter 6

Results

In this chapter, we present the results for the experiments we designed in Chapter 5. Objective metrics were used to measure intelligibility and phoneme preservation of our spect-to-spect and spect-to-wave W2S models. Naturalness and intelligibility scores were also measured using a human listening test. We assessed streaming performance by measuring the lowest possible latency under the condition that there are no streaming artifacts and that RTF ≤ 1.

6.1 Automatic Evaluation

In this sections we present the objective evaluations of our models. Transcriptions were generated for the converted test set using an ASR model. WER and CER were calculated between predicted and ground truth transcriptions. For PA and PER, we used MFA to find phoneme sequences according to the predicted transcriptions. PA is the accuracy of the predicted grid-phoneme sequence. PER is calculated over the collapsed phoneme sequence using Levenshtein distance.

6.1.1 Spectrogram to Waveform

We start by presenting the results obtained with the spect-to-wave approach, which we operationalized using two model types, WaveGrad and DiffWave. WER, PA, PER and CER scores for both model types can be found in Table 6.1. We can clearly see that WaveGrad and DiffWave (both with 6 and with 50 iterations) perform considerably worse than the baseline model in terms of intelligibility. WaveGrad even has a WER higher than 100% which means that the amount of insertions, deletions and substitutions done to revert the predicted transcription to the ground truth transcription is bigger than the amount of words in the ground truth transcription; this can happen if the predicted transcription contains more words than the ground truth transcription. We can also see that DiffWave performs better than WaveGrad on all metrics, indicating that DiffWave better preserves the information contained in the ground truth utterance, even though it uses much less reverse diffusion iterations than WaveGrad at inference time. We will see in Section 6.1.2 that both WaveGrad and DiffWave are less competitive than DiffVC. For this reason we did not explore their performance on the Whispp dataset.

6.1.2 Spectrogram to Spectrogram

We implemented the spect-to-spect approach using multiple adaptations of a diffusion-based voice conversion model [Popov et al., 2021a]. DiffVC-W2S is a decoder trained with with whispered spectrograms using DTW, DiffVC-S2S denotes a decoder trained with only voiced

Model	WER (%)	PA (%)	PER (%)	CER (%)
Ground Truth	11.9	8.6	7.7	3.2
Baseline	**65.9**	**53.4**	**57.2**	**34.3**
WaveGrad (1000)	114.2	80.3	83.1	73.3
DiffWave with Deep Upsampler (50)	82.3	61.4	67.4	45.5
DiffWave with Deep Upsampler (6)	83.3	62.9	68.1	46.1

Table 6.1: This table contains word error rate (WER), phoneme accuracy (PA), phoneme error rate (PER) and character error rate (CER) for the spectrogram to waveform architectures trained on wTIMIT. The top row contains scores for QuartzNet generated with ground truth voiced utterances as described in Section 5.3.1. The baseline is the Whispp model PA is calculated over gridphoneme sequences, which contain phonemes for each frame in a spectrogram, while PER is calculated with collapsed phoneme sequences which can differ in length. The numbers behind the model names (1000, 50, 6) are the amount if diffusion steps used for each model.

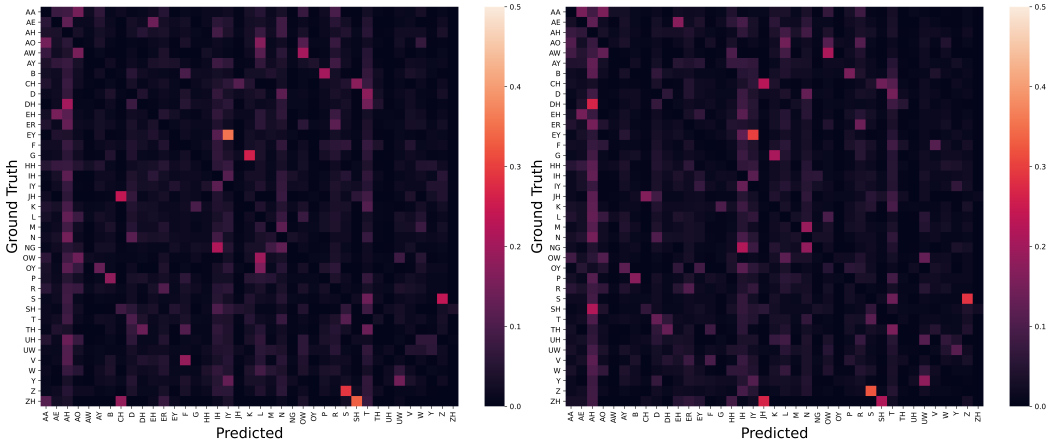
spectrograms and DiffVC-W2P is a model where the encoder is swapped for a whisper to gridphoneme (W2P) model. The decoder in DiffVC-W2P is trained with ground truth average voice representations. This ground truth decoder is DiffVC-GT. We generated scores for DiffVC-GT with ground truth average phoneme sequences, consistent with how it was trained[1].

Table 6.2 shows the results of the wTIMIT-trained DiffVC models tested on wTIMIT. DiffVC-W2P clearly performs best on all recorded metrics. DiffVC-S2S also has better PA and PER compared to the baseline, but worse WER and CER. DiffVC-W2S scores the worst on all recorded metrics. We constructed confusion matrices to analyze the phoneme error of both the baseline model and DiffVC-W2P, by showing what phonemes were often mistaken for others in our test set (Figure 6.1). Both matrices show a lot of similarities. For example, both models often make the mistake of transforming a Z into an S and vice versa. This also happens for $ZH \rightarrow SH$, but not the opposite. Other mistakes that happen frequently for both models are $G \rightarrow K$, $EY \rightarrow IY$ and $\{CH,\ D\} \rightarrow T$. All pairs of phonemes sound very similar, these errors can thus be expected. However, there are some errors that seem unintuitive. For example, the heat map shows that of all the times S was mistaken for something else, it was mistaken for T 10% of the times by the baseline model (Figure 6.1a). Also, AO was mistaken for L 15% of the times AO was mistaken for something else. DiffVC-W2P (Figure 6.1b) appears to have less unintuitive errors.

Model	WER (%)	PA (%)	PER (%)	CER (%)
Ground Truth	11.9	8.6	7.7	3.2
W2P	-	23.5	34.7	-
Baseline	65.9	53.4	57.2	34.3
DiffVC-W2S	81.0	55.1	59.4	43.3
DiffVC-S2S	72.0	51.4	55.3	37.9
DiffVC-W2P	**59.8**	**44.5**	**47.4**	**27.5**
DiffVC-GT	30.3	29.2	25.7	12.6

Table 6.2: DiffVC - Multiple Implementations (Trained on wTIMIT, tested on wTIMIT). Diffusion models used 6 timesteps for synthesis. The W2P column contains ground truth measurements for the whisper to gridphoneme model which were calculated using ground truth whispered utterances.

[1]DiffVC-GT cannot be used in a production setting as we do not have ground truth average voice spectrograms at inference.



(a) (b)

Figure 6.1: Phoneme confusion matrices for the Whispp baseline (left) and DiffVC-W2P (right), both trained on wTIMIT.

We trained and tested our models on Whispp data as well (Table 6.3). We now see that the Whispp baseline performs best on most metrics, except PA, which is best for DiffVC-W2P. WER, PER and CER scores for DiffVC-W2P are very close to those of the baseline model, indicating that the difference in performance is very small. Interestingly, the baseline performs better on Whispp data and DiffVC-W2P performs best on wTIMIT data. This could be due to the fact that voice conversion models are better able to capture the Singaporean accent present in wTIMIT. Just as in Table 6.2, we see that DiffVC-W2P is followed by DiffVC-S2S and DiffVC-W2S in performance. Furthermore, The error rates in this table are the lowest, which indicates that all models perform better on Whispp data than on wTIMIT data. This is expected after taking into account the data features (Table 5.1) and the ground truth measurements.

Model	**WER (%)**	**PA (%)**	**PER (%)**	**CER (%)**
Ground Truth	7.5	3.3	3.8	1.8
W2P	-	22.7	29.0	-
Baseline	**33.6**	32.2	**31.3**	**14.2**
DiffVC-W2S	60.2	47.6	50.4	30.1
DiffVC-S2S	46.0	39.8	42.3	22.3
DiffVC-W2P	36.0	**30.8**	32.2	16.0
DiffVC-GT	19.2	18.6	15.0	9.0

Table 6.3: DiffVC - Multiple Implementations (Trained on Whispp dataset, tested on Whispp dataset)

Finally, Table 6.4 shows the generalization of the models to an unseen dataset. These models were trained on wTIMIT and tested on the Whispp dataset. The baseline clearly performs best on WER, but is matched in performance by DiffVC-W2P. DiffVC-S2S and DiffVC-W2S perform worse, consistent with Tables 6.2 and 6.3. The baseline and Diff-W2P consistently perform better on Whispp data than on wTIMIT data. As explained in Section 5.1, the Whispp dataset was created with higher quality equipment, which could magnify this effect. Indeed the first row of tables 6.4 and 6.3, which shows ground truth QuartzNet scores, shows that ground truth WER, PER and CER are all lower for the Whispp data than for wTIMIT. This has the biggest effect on DiffVC-W2P model because it relies heavily on phoneme clarity. Nevertheless, the models obviously generalize quite well.

Model	WER (%)	PA (%)	PER (%)	CER (%)
Ground Truth	7.5	3.3	3.8	1.8
W2P	-	22.7	29.0	-
Baseline	**58.6**	**50.2**	**52.0**	**31.1**
DiffVC-W2S	79.0	59.1	62.4	43.8
DiffVC-S2S	77.4	57.3	61.0	43.3
DiffVC-W2P	61.0	**50.3**	**52.1**	**31.0**
DiffVC-GT	55.1	41.0	43.5	32.2

Table 6.4: DiffVC - Multiple Implementations (Trained on wTIMIT, tested on Whispp dataset)

In all the tables, we see that the DiffVC-GT and DiffVC-W2P models perform better than the other DiffVC models. The difference is that DiffVC-GT and DiffVC-W2P were trained with ground truth average voice spectrograms instead of spectrograms predicted with an encoder. This indicates that the average voice encoding part is the greatest bottleneck in the process. The inputs for DiffVC-GT and DiffVC-W2P are both constructed with ground truth average phonemes instead of predicted ones. Predicted average voice spectrograms are bound to be more inaccurate than ground truth average phonemes. This could be the reason why DiffVC-S2S and DiffVC-W2S perform worse.

6.2 Human Evaluation

We conducted a listening test (see Section 5.3.2) that gathers human judgments of intelligibility and naturalness on our three best performing models: DiffVC-S2S, DiffVC-W2S and DiffVC-W2P[2]. The baseline was also included so it could be compared against our models. 19 participants completed the test. However, one entry was discarded as this participant finished the test in three minutes, which we do not consider sufficient to faithfully complete it. Additionally, four participants did not correctly order the samples in the warm-up rounds, implicating that these submissions could not be taken into account when calculating the final scores. This left 14 submissions that could be used for evaluation. We used Eq. 5.1 to calculate average ranks for each model. The ranks can be found in Table 6.5.

Model	Naturalness	Intelligibility
Baseline	2.70 ± 1.14	1.97 ± 0.92
DiffVC-W2S	2.58 ± 1.07	2.53 ± 1.03
DiffVC-S2S	**2.31 ± 1.07**	**1.97 ± 0.96**
DiffVC-W2P*	2.41 ± 1.16	3.53 ± 0.75

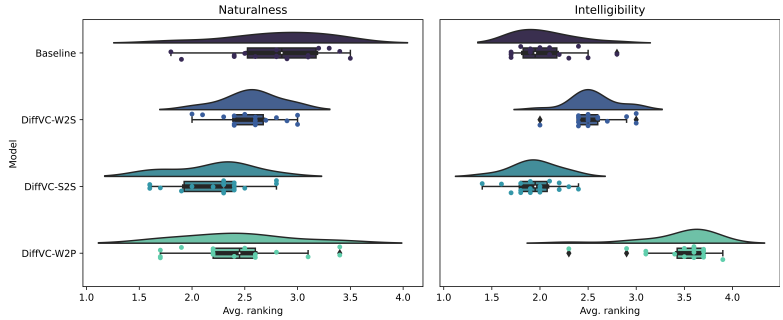
Table 6.5: Ranking scores, the lower the better. The lowest score signifies that the model was given a higher rank more often than the other models. (*) Ranking scores for DiffVC-W2P are invalid (see Appendix B.1 for details).

We can see that DiffVC-S2S was ranked the best on both naturalness and intelligibility. Furthermore, all three DiffVC-based models have higher naturalness than the Whispp baseline. We expect that this is a result of the conditioning on speaker features in the decoder, as this helps the model synthesize more natural speech. For the intelligibility scores, we notice that

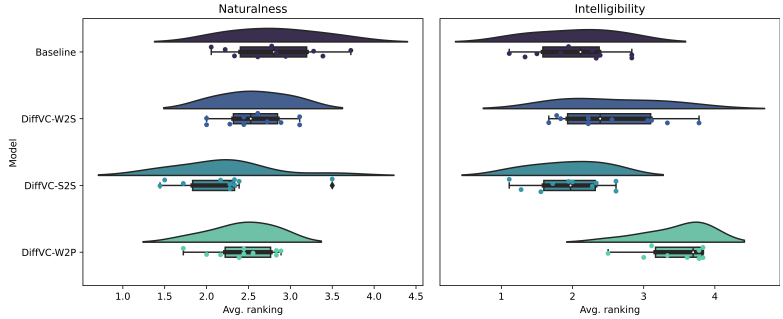
[2]The samples in the test representing DiffVC-W2P were generated incorrectly. See Appendix B.1 for details.

DiffVC-W2S and DiffVC-W2P ranked lower on average than the Whispp baseline, and that the average rank of the baseline model is very close to that of DiffVC-S2S.

To explore the test scores in more depth, we visualized the distribution of the model rankings using violin plots (Figure 6.2). Figure 6.2a shows the average rank each participant gave to the models. Figure 6.2b shows the distribution of how the models were ranked on average for each set. We notice that the plotted distributions agree with the scores in Table 6.5 in terms of the averages. We can see from the box plots underneath the distributions that the participants slightly rated DiffVC-S2S the best on average in terms of intelligibility. These distributions are more spread out in Figure 6.2b (intelligibility), signaling that there was variation between sample sets. This can be explained by the fact that the models are not perfectly consistent in the quality of their synthesis. For the naturalness scores, DiffVC-S2S scored the best in most sets. The participants also rated DiffVC-S2S best in terms of naturalness. However, the naturalness scores given to DiffVC-W2P are very close. It is possible that the lower intelligibility of DiffVC-W2P influenced the participants' judgment of naturalness.



(a) Average over participants*.



(b) Average over sets*.

Figure 6.2: The distribution of test scores when averaged over participants (top) and model sets (bottom). (*) Ranking scores for DiffVC-W2P are invalid (see Appendix B.1 for details).

To study the agreement between raters we employ Kendall's W. This metric is used for calculating agreement between raters that have given ordinal categories to a set of rankable items following Eq. 5.2. The W values are presented in Table 6.6. Following our interpretation table (Table 5.2), we see that agreement between participants was generally weak for the naturalness rankings and moderate for the intelligibility rankings. This is expected as naturalness is more subjective than intelligibility and thus harder to classify unanimously. However, moderate agreement on intelligibility signifies that ranking utterances based on intelligibility is also a difficult and subjective task. We also see that DiffVC-S2S has the highest agreement for naturalness and the lowest agreement for intelligibility. This tells us that participants agreed the most on the naturalness rank for DiffVC-S2S, but found it harder to rank DiffVC-S2S in terms of intelligibility. The converse is true for the baseline and DiffVC-W2S, which apparently were harder to rank in terms of naturalness than intelligibility.

Model	Nat. Ranking	Int. Ranking	Nat. W	Int. W
Baseline	2.7±1.14	1.97±0.92	0.255	0.566
DiffVC-W2S	2.58±1.07	2.53±1.03	0.188	0.533
DiffVC-S2S	2.31±1.07	1.97±0.96	0.374	0.362
DiffVC-W2P*	2.41±1.16	3.53±0.75	0.157	0.4

Table 6.6: Agreement between participants calculated using Kendall's W. The listening test scores are repeated for reference. Naturalness (nat.) and intelligibility (int.) are abbreviated for readability. (*) Ranking scores for DiffVC-W2P are invalid (see Appendix B.1 for details).

6.3 Streaming

In this section we present the results of our streaming research. We first measured the real-time factor of the model, a metric of inference speed (see Section 5.4), at different chunk sizes, with both overlap strategies introduced in Figure 4.7. We only tested DiffVC-S2S as this was our best model[3]. We used 6 reverse diffusion iterations to be consistent with the previous section and because this produced stable results. Table 6.7 shows the computational and structural latency (see Section 5.4) and the RTF. Only the improved overlapping strategy was used due to the benefits mentioned in Section 4.3. The table shows that the lowest chunk size we can achieve out of the box is 16 (256ms), with an overlap size of 14 mel-frames (224ms). This setting produces samples that do not contain noticeable border artifacts. The RTF for this setting is 0.446, which means our model synthesizes samples 2.24 times faster than real-time.

Settings		Results			
Chunk Size	Overlap Size	Computational Lat. (ms)	Structural Lat. (ms)	Total Lat. (ms)	RTF
20	8	185.3	384	533.3	0.416
17	10	184.1	352	536.2	0.427
16	10	184.5	336	**520.5**	0.446
15	14	180.2	352	532.2	0.389
4	8	64.0	192	216	0.333

Table 6.7: RFT and Latency (Lat.). The bottom row contains the measurements for the Whispp baseline. We used a random sample with a duration of 4.064 seconds to gather these results. All displayed settings did not contain any border artifacts with the improved overlapping strategy. We cannot go lower than the lowest total latency (520.5ms) because this will introduce border artifacts.

The total latency for DiffVC-S2S when used in an end-to-end streaming framework is (184.5 + 336 =) 520.5ms. This is twice as slow as the baseline. That difference is mostly a result of the large chunk size required, as the RTF is only marginally bigger. The model was trained with chunks of 128 frames (2048ms), this shows that it generalizes well on shorter utterances. The final latency of 520.5ms does not take external factors into account, such as the time it takes for a chunk to reach the model or the time it takes to transform a spectrogram chunk into a waveform using a vocoder. However, these external factors contributing to the latency are outside the control of the diffusion model and are therefore not considered. We can conclude that although DiffVC-S2S is better than the baseline in terms of naturalness and highly competitive with the baseline in terms of intelligibility—as indicated by human judgments—its structure has to be adapted before it can be employed in a streaming setting.

[3]See footnote 2.

Chapter 7

Conclusion

In this work, we adapted various diffusion models, a very powerful and promising family of generative models, for the task of whisper to speech (W2S) conversion. Converting a whispered utterance to a voiced utterance is not straightforward due to missing information in the whispered utterance (see Section 2.5). This information has to be inferred by a model without altering the phonetic content of a whispered utterance. We adapted two types of architectures: *spect-to-wave* and *spect-to-spect*, tested them and compared them to our baseline. Among the spect-to-wave architectures, we showed showed that DiffWave performs better than WaveGrad in terms of intelligibility, measured through WER, PA, PER and CER. However, both model architectures did not reach the baseline in terms of objective evaluation. For the spect-to-spect route, we adapted a diffusion model built for voice conversion (DiffVC) for the purpose of W2S and showed that a DiffVC model where the encoder is swapped for a whisper to grid-phoneme model (DiffVC-W2P) worked best compared to all our other approaches. Benefits of the DiffVC-W2P approach are that both the encoder and the decoder do not require paired data to be trained. DTW is therefore unnecessary. Furthermore, the decoder is conditioned on speaker characteristics, enabling it to produce very natural speech that sounds exactly like a target speaker. Benefits of diffusion models in general are that they are non-autoregressive, making them very fast, do not not require unstable adversarial training and produce high fidelity samples through a process called Langevin dynamics. The objective metrics showed that DiffVC-W2P performs better than the baseline in terms of all objective metrics when trained on wTIMIT data, which is publicly available. When trained on Whispp data, the baseline performs slightly better on WER and CER and equally well on PA and PER. A possible explanation for the different results for different datasets is that wTIMIT contains strong accents. DiffVC-based models could be better at reconstructing accented utterances because they are conditioned on speaker characteristics. Our subjective listening test showed that all our DiffVC adaptations produced more natural sounding speech than the baseline model. Furthermore, DiffVC-S2S scored higher in terms of human-evaluated intelligibility than the baseline. DiffVC-W2P was not included in the listening test (see Section B.1), but objective evaluation indicates that it probably would have performed even better than DiffVC-S2S. We conclude that diffusion models can be very competitive for W2S compared to the baseline. Especially the spect-to-spect route, which performed much better in terms of both naturalness and intelligibility compared to the spect-to-wave route. We expect that both seeing W2S as a voice conversion problem and the use of diffusion architectures caused the high competitiveness with the Whispp baseline. We discuss how model performance could be improved further in Section 7.1.

We also tested how well our best model would perform in a hypothetical streaming setup. We achieved a total latency of approximately half a second, which is quite fast for a model that

contains attention modules and was not adapted for streaming purposes. However, the baseline model is twice as fast (Table 6.7). Although this currently is not good enough for production purposes, we think that when modifying the model architecture, a latency acceptable for a real streaming setting is achievable with diffusion models. We propose model adaptations that could reduce latency in Section 7.1. As the main goal of this work was to perform whisper to speech conversion with diffusion models, we considered it out of scope to make architectural changes to our models to help latency reduction. In conclusion, when more work on diffusion-based W2S conversion is done, we expect that diffusion models will become state of the art in yet another promising field of artificial intelligence.

7.1 Future Research

This section presents possible future research directions. These directions could all improve the results of the models presented in this work. Due to time constraints, these additional routes were never explored. A first possible improvement is to consider inference during training. Chen et al. [2022] added a loss term to the standard DPM objective that measured the distance between a synthetic sample $\hat{\mathbf{x}}_0$ and the actual sample $\mathbf{x}_0$. They found that this improved their results, especially for low numbers of timesteps t. We made confusion matrices that showed what phonemes were often confused for others (Figure 6.1). Hypothetically, we could define a language model between the encoder and the decoder in DiffVC-based models that helps predicting what phonemes often appear in sequence. For example, *v* is often wrongly classified as *f*. The knowledge that *v* cannot appear in some words, but *f* can, could help reduce the phoneme error and thus also WER and CER. W2P models could be especially suitable for this, as they predict actual phoneme sequences. A language model could greatly increase its accuracy. The ground truth DiffVC decoder already performs much better than the baseline in this work. If most errors could be removed from a W2P model, we would be able to produce even more intelligible samples. To help the encoder of a DiffVC model better generalize between whispered and voiced utterances, it could be trained with both whispered en voiced spectrograms. This may be a way to improve the DiffVC-S2S model. Furthermore, we could design a more sophisticated teacher-student setup where we train an encoder on whispered spectrograms, and another encoder on spoken spectrograms. We could use the learning signal of the spoken encoder to *teach* the whispered encoder to improve its average phoneme predictions. To improve our models for streaming, we need to make modifications to our model architectures. We could start by decreasing the spectrogram size that is used to train the model. This reduces the receptive field size for the attention modules, forcing it to focus on features in a shorter sequence. A more rigorous adaptation would be to mask out all the attention weights of forward-looking connections during training the attention modules. This would further decrease structural latency, but possibly also model performance.

Bibliography

R. J. Ingham, Anne K. Bothe, Erin Michelle Jang, Lauren Yates, John Cotton, and Irene Seybold. Measurement of speech effort during fluency-inducing conditions in adults who do and do not stutter. Journal of speech, language, and hearing research : JSLHR, 52 5: 1286–301, 2009.

Tomoki Toda and Kiyohiro Shikano. Nam-to-speech conversion with gaussian mixture models. In INTERSPEECH, pages 1957–1960, 09 2005. doi: 10.21437/Interspeech.2005-611.

Harshit Malaviya, Jui Shah, Maitreya Patel, Jalansh Munshi, and Hemant A. Patil. Mspecnet : Multi-domain speech conversion network. In ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 7764–7768, 2020. doi: 10.1109/ICASSP40776.2020.9052966.

Taeksoo Kim, Moonsu Cha, Hyunsoo Kim, Jung Kwon Lee, and Jiwon Kim. Learning to discover cross-domain relations with generative adversarial networks, 2017.

Teng Gao, Jian Zhou, Huabin Wang, Liang Tao, and Hon Keung Kwan. Attention-guided generative adversarial network for whisper to normal speech conversion, 2021.

Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution, 2020.

Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics, 2015.

Ruojin Cai, Guandao Yang, Hadar Averbuch-Elor, Zekun Hao, Serge Belongie, Noah Snavely, and Bharath Hariharan. Learning gradient fields for shape generation, 2020.

Gautam Mittal, Jesse Engel, Curtis Hawthorne, and Ian Simon. Symbolic music generation with diffusion models, 2021.

Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis, 2021.

Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In International Conference on Learning Representations, 2021. URL `https://openreview.net/forum?id=PxTIG12RRHS`.

Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. arXiv preprint arXiv:2204.06125, 2022.

Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, S Sara Mahdavi, Rapha Gontijo Lopes, et al. Photorealistic text-to-image diffusion models with deep language understanding. arXiv preprint arXiv:2205.11487, 2022.

Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. arXiv preprint arXiv:2205.14217, 2022.

Nanxin Chen, Yu Zhang, Heiga Zen, Ron J. Weiss, Mohammad Norouzi, and William Chan. Wavegrad: Estimating gradients for waveform generation, 2020.

Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. Diffwave: A versatile diffusion model for audio synthesis, 2021.

Vadim Popov, Ivan Vovk, Vladimir Gogoryan, Tasnima Sadekova, Mikhail Kudinov, and Jiansheng Wei. Diffusion-based voice conversion with fast maximum likelihood sampling scheme, 2021a.

Jianwei Zhang, Suren Jayasuriya, and Visar Berisha. Restoring degraded speech via a modified diffusion model. Interspeech 2021, Aug 2021. doi: 10.21437/interspeech.2021-1889. URL `http://dx.doi.org/10.21437/Interspeech.2021-1889`.

Thomas Rossing. Springer Handbook of Acoustics. Springer, 2007.

H. Nyquist. Certain topics in telegraph transmission theory. Transactions of the American Institute of Electrical Engineers, 47(2):617–644, 1928. doi: 10.1109/T-AIEE.1928.5055024.

Michael McAuliffe, Michaela Socolof, Sarah Mihuc, Michael Wagner, and Morgan Sonderegger. Montreal forced aligner: Trainable text-speech alignment using kaldi. In INTERSPEECH, pages 498–502, 08 2017. doi: 10.21437/Interspeech.2017-1386.

Ronald Newbold Bracewell and Ronald N Bracewell. The Fourier transform and its applications, volume 31999. McGraw-Hill New York, 1986.

J. Volkmann, S. S. Stevens, and E. B. Newman. A scale for the measurement of the psychological magnitude pitch. The Journal of the Acoustical Society of America, 8(3):208–208, 1937. doi: 10.1121/1.1901999. URL `https://doi.org/10.1121/1.1901999`.

Ben Gold, Nelson Morgan, and Dan Ellis. Speech and Audio Signal Processing: Processing and Perception of Speech and Music. John Wiley & Sons, 2011. ISBN 1118142918.

D. Griffin and Jae Lim. Signal estimation from modified short-time fourier transform. In ICASSP '83. IEEE International Conference on Acoustics, Speech, and Signal Processing, volume 8, pages 804–807, 1983. doi: 10.1109/ICASSP.1983.1172092.

Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio, 2016.

Kundan Kumar, Rithesh Kumar, Thibault de Boissiere, Lucas Gestin, Wei Zhen Teoh, Jose Sotelo, Alexandre de Brebisson, Yoshua Bengio, and Aaron Courville. Melgan: Generative adversarial networks for conditional waveform synthesis, 2019.

Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis, 2020.

Nichole Houle and Susannah V. Levi. Acoustic differences between voiced and whispered speech in gender diverse speakers. The Journal of the Acoustical Society of America, 148(6):4002–4013, 2020. doi: 10.1121/10.0002952.

P. Kenny, G. Boulianne, and P. Dumouchel. Eigenvoice modeling with sparse training data. IEEE Transactions on Speech and Audio Processing, 13(3):345–354, 2005. doi: 10.1109/TSA.2004.840940.

Mickael Rouvier, Pierre-Michel Bousquet, and Benoit Favre. Speaker diarization through speaker embeddings. In 2015 23rd European Signal Processing Conference (EUSIPCO), pages 2082–2086, 2015. doi: 10.1109/EUSIPCO.2015.7362751.

Xiangang Li and Xihong Wu. Modeling speaker variability using long short-term memory networks for speech recognition. In INTERSPEECH, 2015.

Ye Jia, Yu Zhang, Ron J. Weiss, Quan Wang, Jonathan Shen, Fei Ren, Zhifeng Chen, Patrick Nguyen, Ruoming Pang, Ignacio Lopez Moreno, and Yonghui Wu. Transfer learning from speaker verification to multispeaker text-to-speech synthesis, 2019.

Shuai Wang, Yanmin Qian, and Kai Yu. What does the speaker embedding encode? In INTERSPEECH, 2017.

Shehla Abbas, Mohamed Mosbah, Akka Zemmari, and Université Bordeaux. Itu-t recommendation g.114, "one way transmission time. In In International Conference on Dynamics in Logistics 2007 (LDIC 2007), Lect. Notes in Comp. Sciences. Springer-Verlag, 1996.

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014.

Yuma Koizumi, Heiga Zen, Kohei Yatabe, Nanxin Chen, and Michiel Bacchiani. Specgrad: Diffusion probabilistic model based neural vocoder with adaptive noise spectral shaping. arXiv preprint arXiv:2203.16749, 2022.

Kaizhi Qian, Yang Zhang, Shiyu Chang, Xuesong Yang, and Mark Hasegawa-Johnson. Autovc: Zero-shot voice style transfer with only autoencoder loss, 2019.

Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2014.

Diederik P. Kingma, Danilo J. Rezende, Shakir Mohamed, and Max Welling. Semi-supervised learning with deep generative models, 2014.

Marius Cotescu, Thomas Drugman, Goeric Huybrechts, Jaime Lorenzo-Trueba, and Alexis Moinet. Voice conversion for whispered speech synthesis. IEEE Signal Processing Letters, 27:186–190, 2020. ISSN 1558-2361. doi: 10.1109/lsp.2019.2961213. URL `http://dx.doi.org/10.1109/LSP.2019.2961213`.

Amazon.com Inc. Alexa is now even smarter, Sep 2018. URL `https://press.aboutamazon.com/news-releases/news-release-details/alexa-now-even-smarter-new-features-help-make-everyday-life-more/`.

William Feller. On the theory of stochastic processes, with particular reference to applications. In Proceedings of the [First] Berkeley Symposium on Mathematical Statistics and Probability, pages 403–432. University of California Press, 1949.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020.

Albert Einstein. On the movement of small particles suspended in stationary liquids required by the molecular kinetic theory of heat. Annalen der Physik, 17(549-560):1, 1905.

Paul Langevin. Sur la théorie du mouvement brownien. Comptes Rendus Mathématique, 146: 530–533, 1908.

George E Uhlenbeck and Leonard S Ornstein. On the theory of the brownian motion. Physical review, 36(5):823, 1930.

Max Welling and Yee W Teh. Bayesian learning via stochastic gradient langevin dynamics. In Proceedings of the 28th international conference on machine learning (ICML-11), pages 681–688. Citeseer, 2011.

Qiang Liu, Jason D. Lee, and Michael I. Jordan. A kernelized stein discrepancy for goodness-of-fit tests and model evaluation, 2016.

Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. Optimization by simulated annealing. science, 220(4598):671–680, 1983.

Radford M Neal. Annealed importance sampling. Statistics and computing, 11(2):125–139, 2001.

Yang Song. Generative modeling by estimating gradients of the data distribution, 2021. URL `https://yang-song.github.io/blog/2021/score/`.

Aapo Hyvärinen. Estimation of non-normalized statistical models by score matching. Journal of Machine Learning Research, 6(24):695–709, 2005. URL `http://jmlr.org/papers/v6/hyvarinen05a.html`.

Pascal Vincent. A connection between score matching and denoising autoencoders. Neural computation, 23(7):1661–1674, 2011.

Yang Song, Sahaj Garg, Jiaxin Shi, and Stefano Ermon. Sliced score matching: A scalable approach to density and score estimation, 2019.

Brian DO Anderson. Reverse-time diffusion equation models. Stochastic Processes and their Applications, 12(3):313–326, 1982.

Eckhard Platen and Nicola Bruti-Liberati. Numerical solution of stochastic differential equations with jumps in finance, volume 64. Springer Science & Business Media, 2010.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Deep Learning. The MIT Press, 2016. ISBN 0262035618.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, Department of Computer Science, 2009.

Fisher Yu, Ari Seff, Yinda Zhang, Shuran Song, Thomas Funkhouser, and Jianxiong Xiao. Lsun: Construction of a large-scale image dataset using deep learning with humans in the loop, 2016.

Alex Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models, 2021.

Nanxin Chen, Yu Zhang, Heiga Zen, Ron J. Weiss, Mohammad Norouzi, Najim Dehak, and William Chan. Wavegrad 2: Iterative refinement for text-to-speech synthesis, 2021.

Soroush Mehri, Kundan Kumar, Ishaan Gulrajani, Rithesh Kumar, Shubham Jain, Jose Sotelo, Aaron Courville, and Yoshua Bengio. Samplernn: An unconditional end-to-end neural audio generation model, 2017.

Nal Kalchbrenner, Erich Elsen, Karen Simonyan, Seb Noury, Norman Casagrande, Edward Lockhart, Florian Stimberg, Aaron van den Oord, Sander Dieleman, and Koray Kavukcuoglu. Efficient neural audio synthesis, 2018.

Vadim Popov, Ivan Vovk, Vladimir Gogoryan, Tasnima Sadekova, and Mikhail Kudinov. Grad-tts: A diffusion probabilistic model for text-to-speech, 2021b.

Vincent Dumoulin, Ethan Perez, Nathan Schucher, Florian Strub, Harm de Vries, Aaron Courville, and Yoshua Bengio. Feature-wise transformations. Distill, 3(7):e11, 2018.

Boon Pang Lim. Computational Differences between Whispered and Non-Whispered Speech. PhD thesis, University of Illinois, 01 2011.

Samuel Kriman, Stanislav Beliaev, Boris Ginsburg, Jocelyn Huang, Oleksii Kuchaiev, Vitaly Lavrukhin, Ryan Leary, Jason Li, and Yang Zhang. Quartznet: Deep automatic speech recognition with 1d time-channel separable convolutions. In ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 6124–6128. IEEE, 2020.

Oleksii Kuchaiev, Jason Li, Huyen Nguyen, Oleksii Hrinchuk, Ryan Leary, Boris Ginsburg, Samuel Kriman, Stanislav Beliaev, Vitaly Lavrukhin, Jack Cook, et al. Nemo: a toolkit for building ai applications using neural modules. arXiv preprint arXiv:1909.09577, 2019.

Vladimir I Levenshtein et al. Binary codes capable of correcting deletions, insertions, and reversals. In Soviet physics doklady, volume 10, pages 707–710. Soviet Union, 1966.

Sławomir Zieliński. On some biases encountered in modern listening tests. Proceedings of Spatial Audio & Sensory Evaluation Techniques, 04 2006.

Berrak Sisman, Junichi Yamagishi, Simon King, and Haizhou Li. An overview of voice conversion and its challenges: From statistical modeling to deep learning. IEEE/ACM Transactions on Audio, Speech, and Language Processing, 29:132–157, 2020.

Yadolah Dodge, David Cox, and Daniel Commenges. The Oxford dictionary of statistical terms. Oxford University Press on Demand, 2006.

Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. Journal of the American Statistical Association, 32(200):675–701, 1937.

Sarbast Moslem, Omid Ghorbanzadeh, Thomas Blaschke, and Szabolcs Duleba. Analysing stakeholder consensus for a sustainable transport development decision by the fuzzy ahp and interval ahp. Sustainability, 11:3271, 06 2019. doi: 10.3390/su11123271.

Zehua Chen, Xu Tan, Ke Wang, Shifeng Pan, Danilo Mandic, Lei He, and Sheng Zhao. Infergrad: Improving diffusion models for vocoder by considering inference in training. In ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pages 8432–8436. IEEE, 2022.

Appendix A

A.1 ELBO Definition

We want to approximate the true reverse data distribution $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$. Because it is not known, we approximate it with a neural network $p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ parameterized by $\boldsymbol{\theta}$. The optimal $\boldsymbol{\theta}$ should minimize the negative log-likelihood of the model. We can use the ELBO for this which is of the standard form:

$$\mathbb{E}[\log p(x)] \geq \sum_h q(h|x) \log \frac{p(x,h)}{q(h|x)} \tag{A.1}$$

$$= \sum_h q(h|x) \log p(x,h) - \sum_h q(h|x) \log p(h|x) \tag{A.2}$$

$$= \mathbb{E}_q[\log p(x,h) - \log q(h|x)] \tag{A.3}$$

$$= \mathbb{E}_q\left[\log \frac{p(x,h)}{q(h|x)}\right]. \tag{A.4}$$

In the equation above, h is a latent variable, such as the intermediate steps in our diffusion processes ($\mathbf{x}_t$) and $q(h|x)$ can be seen as an approximation of the true distribution $p(h|x)$. Note that it is not important which is the true distribution in the ELBO as the difference between the approximate distribution and the true distribution will be minimized anyway. Only the parameterized distribution will change during optimization. When we replace the standard variables in the ELBO with the variables from our DPM formulation we get

$$\mathbb{E}[\log p_{\boldsymbol{\theta}}(\mathbf{x}_0)] \geq \mathbb{E}_q\left[\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right]. \tag{A.5}$$

We then flip the sign on both sides so that we have an expression for the negative log-likelihood, which is more convenient for computation:

$$\mathbb{E}[-\log p_{\boldsymbol{\theta}}(\mathbf{x}_0)] \leq \mathbb{E}_q\left[-\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right]. \tag{A.6}$$

As a final step, we extract $p(\mathbf{x}_T)$ from $p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T})$ because it does not need to be approximated (Eq. 3.8). The final variational bound becomes:

$$\mathbb{E}[-\log p_{\boldsymbol{\theta}}(\mathbf{x}_0)] \leq \mathbb{E}_q\left[-\log p(\mathbf{x}_T) - \sum_{t\geq 1} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})}\right]. \tag{A.7}$$

A.2 ELBO Reparameterization

We reparameterize the ELBO from Eq. 3.10 to make it computationally tractable. The reparameterization is adapted from [Sohl-Dickstein et al., 2015] and [Ho et al., 2020]. The critical rewrite is that from $q(\mathbf{x}_t|\mathbf{x}_{t-1})$ which is intractable, to $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ which can be computed. Conditioning on $\mathbf{x}_0$ makes $\tilde{\boldsymbol{\mu}}_t$ in $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I})$ computable in closed form (Eq. 3.14).

$$L = \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t \geq 1} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] \tag{A.8}$$

$$= \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t > 1} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} - \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)}{q(\mathbf{x}_1|\mathbf{x}_0)} \right] \tag{A.9}$$

$$= \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t > 1} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)} \cdot \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} - \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)}{q(\mathbf{x}_1|\mathbf{x}_0)} \right] \tag{A.10}$$

$$= \mathbb{E}_q \left[-\log \frac{p(\mathbf{x}_T)}{q(\mathbf{x}_T|\mathbf{x}_0)} - \sum_{t > 1} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)} - \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) \right] \tag{A.11}$$

$$= \mathbb{E}_q \left[D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T)) + \sum_{t > 1} D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)) - \log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1) \right]. \tag{A.12}$$

Appendix B

B.1 The effect of dropout and batch normalization on DiffVC-W2P

After aggregating and interpreting the results of the human listening test, we found that the samples for the DiffVC-W2P model were generated incorrectly. Namely, dropout layers and batch normalization layers of the whisper to gridphoneme (W2P) model were not set to evaluation mode during inference. The correction caused a performance increase of approximately 50% on our objective metrics (WER, PA, PER, CER). Consequently, DiffVC-W2P is now the best of our models instead of DiffVC-S2S. We rewrote Sections 6.1 and 7 to include our new findings, but the listening test could not be conducted again due to time constraints. Section 6.2 therefore contains results where participants rated DiffVC-S2S as our best model. Nevertheless, we include the results of the listening test in our work because they provide interesting details, and alert the reader whenever results have to be interpreted differently.